

Abstract

In this thesis, we study the distributed minimum set cover problem and its weighted variant. We present two deterministic distributed algorithms in the **LOCAL** model.

For the unweighted set cover problem, where each set contains at most s elements and each element belongs to at most t sets, we present a deterministic $1 + (2 + \epsilon)\lceil \ln s \rceil$ -approximate algorithm with $O(\log s \cdot \log^2 t + \log^* n) \cdot \text{poly}(1/\epsilon)$ round complexity. Previously, Faour et al. [SODA'23] developed a local rounding framework and used it to obtain an $O(\log s)$ -approximate $O(\log s \cdot \log^2 t + \log^* n)$ -round deterministic algorithm. Using the same framework, we improve the approximation ratio from $O(\log s)$ to $1 + (2 + \epsilon)\lceil \ln s \rceil$, at the cost of only a $\text{poly}(1/\epsilon)$ factor in round complexity.

For the weighted variant, we present an algorithm with the same approximation ratio as the unweighted algorithm and $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon) + O(\log t \cdot \log W)$ round complexity. Ghaffari and Grunau [STOC'23] combined two approaches for developing deterministic distributed algorithms, local rounding and network decomposition, to improve results on deterministic maximal independent set and $O(1)$ -approximate maximum matching. We apply this combined approach to the weighted set cover problem: nodes first compute a fractional solution within low-diameter clusters, then apply local rounding to obtain an integral solution. By setting all weights to 1, we additionally obtain an alternative unweighted set cover algorithm with $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon)$ round complexity.

Contents

1	Introduction	1
1.1	Background	1
1.2	Our Contributions	2
2	Preliminaries	4
2.1	Set Cover Problem and Concentration Inequalities	4
2.2	Local Rounding Framework	5
2.3	Clustering	6
3	Unweighted Set Cover	7
3.1	Obtaining Fractional Solutions	9
3.2	Rounding $1/K$ -Integral Solution	11
3.2.1	Analyzing One Round of Local Rounding	14
3.3	Approximation Ratio and Round Complexity	16
4	Weighted Set Cover	21
4.1	Computing $1/K$ -Integral Solution with Clustering	22
4.2	Rounding $1/K$ -Integral Solution	30
4.2.1	Analyzing One Round of Local Rounding	32
4.3	Approximation Ratio and Round Complexity	34
5	Discussion	39
5.1	Future Work	39
	Bibliography	41
A	Appendix	44

List of Algorithms

1	Randomized Set Cover	7
2	Deterministic $O(\log s)$ -Approximate Set Cover (Taken from [FGG ⁺ 23])	8
3	Deterministic $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -Approximate Set Cover	9
4	Deterministic $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -Approximate Weighted Set Cover	22

1 Introduction

The distributed minimum set cover problem has long been one of the fundamental problems in distributed computing. In recent years, significant progress has been made on deterministic distributed algorithms for several core problems, including maximal independent set (MIS), maximal matching, and $O(1)$ -approximate maximum matching. A key breakthrough was the demonstration by Ghaffari and Grunau [GG23] that two previously unrelated techniques, *local rounding* and *network decomposition*, can be combined to surpass the best known deterministic bounds on some of these problems. Our approach is to apply these two ideas to the set cover problem. In this thesis, we present deterministic approximation algorithms for the distributed minimum set cover problem and its weighted version.

1.1 Background

Model. We work with a synchronous message-passing model, referred to as the **LOCAL** model in distributed computing [Pel00, Lin87]. The network is modeled by an undirected graph $G = (V, E)$ with $n = |V|$ nodes, where each node represents a processor with a unique $O(\log n)$ -bit identifier and each edge is a two-way communication link. Initially, nodes have no information about the network topology. However, they may have information about some global parameters, such as a polynomial upper bound on the number of nodes. Two nodes are called neighbors if they are connected by a communication link. In each round, each node may send a message to each of its neighbors. In **LOCAL**, message sizes are unbounded, in contrast to the **CONGEST** model where message sizes are bounded by $O(\log n)$ bits. At the end of the computation, each node decides its output for the given problem.

Network Decomposition. A cluster in a graph $G = (V, E)$ is a subset $C \subseteq V$ such that the subgraph induced by C is connected [Pel00]. A partition of a graph $G = (V, E)$ is a collection of pairwise disjoint clusters $\{S_1, \dots, S_k\}$ where $\cup_{i=1}^k S_i = V$ [Pel00]. A (c, d) -decomposition is a partition of the graph into c blocks where each block only contains non-adjacent clusters and the diameter of each cluster is at most d [ALGP89]. Network decomposition is an important tool for designing deterministic algorithms for several crucial problems in distributed computing such as MIS. In recent years, there have been significant developments in constructing faster decomposition algorithms [RG20, GGR21, GGH⁺23]. Lastly, Ghaffari and Grunau [GG23] proposed an algorithm for a relaxed variant of network decomposition. We use this particular algorithm for the distributed minimum weighted set cover problem.

Local Rounding. Another approach to designing deterministic distributed algorithms is local rounding. With this approach, we start with a valid fractional solution to the given problem; then we gradually mod-

ify this solution in rounds to obtain an integral solution while approximately preserving the quality of the original fractional solution. This technique can be traced back to Hanckowiak, Karonski, and Panconesi [HKP01], who gave the first $\text{poly}(\log n)$ -round deterministic algorithm for the maximal matching problem. Recently, Fischer [Fis20] improved the round complexity of the maximal matching to $O(\log^2 \Delta \cdot \log n)$ using local rounding, where Δ is the maximum degree. For the $(2\Delta - 1)$ edge coloring problem, Fischer, Ghaffari, and Kuhn [FGK17] developed a $\text{poly}(\log n)$ -round algorithm, later improved by Harris [Har19] to $\tilde{O}(\log^2 \Delta \cdot \log n)$. Another related problem, $(\Delta + 1)$ coloring, is also improved to $O(\log^2 \Delta \cdot \log n)$ by the same method by Ghaffari and Kuhn [GK22]. These recent solutions, however, were specific to the problem they solved; the local rounding method was not generalized. Finally, Faour et al. [FGG⁺23] developed a framework for local rounding and demonstrated its effectiveness by designing an $O(\log^2 \Delta \cdot \log n)$ -round deterministic algorithm for the MIS problem.

Unified Approach. Ghaffari and Grunau [GG23] showed that the two approaches, network decomposition and local rounding, can be combined to develop deterministic distributed algorithms that can improve upon the state-of-the-art. The combined approach works as follows: First, nodes partition the graph into low-diameter clusters. Then, nodes compute a fractional solution using only the information from the other nodes within their cluster. Lastly, this fractional solution is rounded to an integral solution using local rounding. With this unified approach, they designed a deterministic MIS algorithm which runs in $\tilde{O}(\log^2 n)$ rounds. Previously, the best known deterministic MIS round complexity had remained in $\Omega(\log^3 n)$ [GGH⁺23]. Similarly, this method also yields a faster deterministic algorithm for $O(1)$ -approximate maximum matching [GG23].

Set Cover. The minimum set cover problem is one of the most fundamental optimization problems in theoretical computer science, and it is closely related to other problems such as the minimum dominating set problem and the minimum vertex cover problem [GJ79, DKM19]. We provide a formal description of the problem and its variants in Section 2.1. The problem has practical applications in computer networks, information retrieval, operations research, and many other fields [CKW10, DKM19]. It is an NP-hard problem [KV12]; furthermore, even computing a $(1 - o(1)) \ln n$ -approximate solution is NP-hard [DS14]. For the distributed variant of the problem, Faour et al. [FGG⁺23] presented a deterministic $O(\log s)$ -approximate set cover algorithm with $O(\log s \cdot \log^2 t + \log^* n)$ round complexity. This is a near-exponential improvement in round complexity over the previous state-of-the-art deterministic algorithm [DKM19]. In this thesis, we improve upon the approximation ratio of [FGG⁺23] and extend the result to the weighted variant.

1.2 Our Contributions

In [FGG⁺23], a deterministic algorithm is given for the distributed set cover problem using the local rounding approach. This algorithm has an approximation ratio of $O(\log s)$ and round complexity $O(\log s \cdot \log^2 t + \log^* n)$. Note that the hidden constant in the $O(\log s)$ term can be large. Our first contribution is to introduce a $(1 + (2 + \epsilon) \lceil \ln s \rceil)$ -approximation algorithm for the distributed set cover problem.

Theorem 1.1. *There is a deterministic distributed algorithm in the LOCAL model that computes a $(1 + (2 + \epsilon) \lceil \ln s \rceil)$ -approximate set cover in $O(\log s \cdot \log^2 t + \log^* n) \cdot \text{poly}(1/\epsilon)$ rounds, where each set has at*

most s elements ($s \geq \max_{v \in V} \deg(v)$), each element is contained in at most t sets ($t \geq \max_{u \in U} \deg(u)$), and $0 < \epsilon \leq 1$.

In the previous subsection, we mentioned that the unified approach of local rounding and network decomposition resulted in faster deterministic algorithms [GG23]. Our second contribution is to give an algorithm for the weighted version of the distributed set cover problem by utilizing this unified approach.

Theorem 1.2. *There is a deterministic distributed algorithm in the LOCAL model that computes a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate weighted set cover in $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon) + O(\log t \log W)$ rounds, where the maximum set weight is at most W ($W \geq \max_{v \in V} w(v)$), each set has at most s elements ($s \geq \max_{v \in V} \deg(v)$), each element is contained in at most t sets ($t \geq \max_{u \in U} \deg(u)$), and $0 < \epsilon \leq 1$.*

We can use Theorem 1.2 to obtain another result for the unweighted version of the problem with the same approximation ratio given in Theorem 1.1.

Corollary 1.3. *There is a deterministic distributed algorithm in the LOCAL model that computes a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate set cover in $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon)$ rounds, where each set has at most s elements ($s \geq \max_{v \in V} \deg(v)$) and $0 < \epsilon \leq 1$.*

Throughout this thesis, our aim is to prove Theorem 1.1 and Theorem 1.2. Chapter 2 provides the necessary preliminaries. Chapter 3 presents the unweighted set cover algorithm, and Chapter 4 presents the weighted algorithm. Finally, Chapter 5 concludes the thesis with a discussion on the algorithms presented.

2 Preliminaries

2.1 Set Cover Problem and Concentration Inequalities

The definitions of the set cover problem, with its weighted and fractional variants, are given below:

Definition 2.1 (Set Cover Problem). *Given a ground set of elements $\mathcal{U}$ and a collection of sets $S = \{S_1, \dots, S_k\}$ where each $S_i \subseteq \mathcal{U}$, find a subset $I \subseteq S$ with minimum cardinality that covers $\mathcal{U}$, i.e., $\bigcup_{S_i \in I} S_i = \mathcal{U}$.*

Definition 2.2 (Weighted Set Cover Problem). *Given a ground set of elements $\mathcal{U}$, a collection of sets $S = \{S_1, \dots, S_k\}$, and a weight function $w : S \rightarrow \mathbb{R}^+$, where each $S_i \subseteq \mathcal{U}$, find a subset $I \subseteq S$ with the minimum total weight that covers $\mathcal{U}$, i.e., $\bigcup_{S_i \in I} S_i = \mathcal{U}$.*

Definition 2.3 (Fractional Set Cover Problem). *Given a ground set of elements $\mathcal{U}$ and a collection of sets $S = \{S_1, \dots, S_k\}$ where each $S_i \subseteq \mathcal{U}$, find a vector $\vec{x} = (x_1, \dots, x_k)$, where $0 \leq x_m \leq 1$ for all $m = 1, \dots, k$, that minimizes $\sum_{m=1}^k x_m$ and covers $\mathcal{U}$, i.e., for all $\mu \in \mathcal{U}$, $\sum_{m|\mu \in S_m} x_m \geq 1$.*

We are interested in the distributed version of the problem. The distributed set cover problem in the LOCAL model is formulated as follows:

Definition 2.4 (Modeling of the Distributed Set Cover, [EGM18], [KMW16]). *Our model uses an undirected bipartite graph $G = (U \sqcup V, E)$ where $n = |U \sqcup V|$. Each element μ in the ground set $\mathcal{U}$ is represented by a node $u \in U$, and each set S_i is represented by a node $s_i \in V$. An edge $\{u, s_i\}$ exists if and only if S_i contains the element μ . For the weighted version, we add a weight function to the model, $w : V \rightarrow [1, \infty)$. Initially, each node in $U \sqcup V$ knows N , s , and t , where N is a polynomial upper bound on n , $s \geq \max_{v \in V} \deg(v)$, and $t \geq \max_{u \in U} \deg(u)$. We can safely assume that s and t are upper-bounded by N . Additionally, for the weighted version, nodes know W where $W \geq \max_{v \in V} w(v)$.*

We should briefly explain why we restrict $s, t \leq N$: Suppose nodes have the information $s > N$ for a problem instance. Since $\max_{v \in V} \deg(v) \leq N$, nodes can agree to use a tighter value $s' = N$ for the upper bound of $\max_{v \in V} \deg(v)$. A similar argument applies for the case where $t > N$.

Next, we define a property of sets containing real numbers.

Definition 2.5 ($1/K$ -Integrality). *A set of values $\{x_1, \dots, x_k\}$ is called $1/K$ -integral if for all i , $x_i = 0$ or $x_i \geq 1/K$.*

We utilize a particular concentration inequality in the weighted set cover algorithm. It is a variation of the Chernoff bound; therefore, we first state the Chernoff bound:

Theorem 2.1 (Chernoff Bound, [MU17, HMN22]). *Let $X_1, \dots, X_k$ be independent random variables with $X_i \in \{0, 1\}$ and $\Pr[X_i = 1] = p_i$ for all $i = 1, \dots, k$. Let $X = \sum_{i=1}^k X_i$. For $\delta > 0$:*

$$\Pr(|X - \mathbb{E}[X]| \geq \delta \mathbb{E}[X]) \leq 2e^{-\frac{\min(\delta, \delta^2) \mathbb{E}[X]}{3}}$$

We adapt this bound to a set of random variables that can only take the values 0 and b where $b > 0$:

Corollary 2.2 (Generalized Version of Chernoff Bound in [HMN22]). *Let $Y_1, \dots, Y_k$ be independent random variables with $Y_i \in \{0, b\}$ where $b > 0$ and $\Pr[Y_i = b] = p_i$ for all $i = 1, \dots, k$. Let $Y = \sum_{i=1}^k Y_i$. Then, for a given $r \geq \mathbb{E}[Y]/\omega$ where $\omega > 1$:*

$$\Pr[|Y - \mathbb{E}[Y]| \geq r] \leq 2e^{-\frac{r}{3\omega b}}$$

Proof. Let $X_i = Y_i/b$ for every i , and $X = \sum_{i=1}^k X_i$. Let $\delta = \frac{r}{\mathbb{E}[Y]} \geq 1/\omega$. Then,

$$\Pr[|Y - \mathbb{E}[Y]| \geq r] = \Pr[|X - \mathbb{E}[X]| \geq \delta \mathbb{E}[X]] \leq 2e^{-\frac{\min(\delta, \delta^2) \mathbb{E}[X]}{3}} \leq 2e^{-\frac{\delta \mathbb{E}[X]}{3\omega}} = 2e^{-\frac{r}{3\omega b}}$$

□

2.2 Local Rounding Framework

In both algorithms we present, local rounding is used to convert a fractional solution to an integral one. First, it is beneficial to review the definitions introduced by the framework.

Definition 2.6 (Label Assignment, Utility Function, Cost Function [FGG⁺23]). *Let $G = (V, E)$ be a graph. A label assignment for G is a function $\ell : V \rightarrow \Sigma$. A fractional label assignment $\lambda : V \rightarrow [0, 1]^{|\Sigma|}$ for G is a function such that for each $v \in V$, $\sum_{\alpha \in \Sigma} \lambda_\alpha(v) = 1$. Let $S \subseteq V$ be a set of vertices. For a label assignment ℓ , let $\ell(S)$ denote the label assignment to the set of nodes S . $\lambda(S)$ is defined similarly for a fractional label assignment λ . Let $\mathcal{L}(S)$ (and $\Lambda(S)$) denote the set of all possible (fractional) label assignments to S . A utility function is a non-negative function $\mathbf{u} : E \times \mathcal{L}(V) \rightarrow \mathbb{R}_{\geq 0}$ such that $\mathbf{u}(\{u, v\}, \ell)$ depends only on the label assignments of u and v . We can further define $\mathbf{u}$ for fractional labelings: $\mathbf{u} : E \times \Lambda(V) \rightarrow \mathbb{R}_{\geq 0}$ where*

$$\mathbf{u}(\{u, v\}, \lambda) = \sum_{(\alpha, \beta) \in \Sigma^2} \lambda_\alpha(u) \cdot \lambda_\beta(v) \cdot \mathbf{u}(\{u, v\}, \ell_{\alpha, \beta}(\{u, v\}))$$

Let $F \subseteq E$ be a set of edges. For a label assignment ℓ , we define $\mathbf{u}(F, \ell) = \sum_{e \in F} \mathbf{u}(e, \ell)$. This definition extends to fractional label assignments by replacing ℓ with λ . The cost function $\mathbf{c}$ is defined similarly to $\mathbf{u}$.

The definitions of utility and cost functions are slightly restrictive for our purposes as we need to use utility and cost functions defined on vertices, not on edges. Fortunately, the framework addresses this limitation.

Lemma 2.3 (Lemma 2.4 of [FGG⁺23]). *Let $G = (V, E)$ be a graph equipped with utility and cost functions $\mathbf{u}$ and $\mathbf{c}$. Suppose the functions can be extended to assign a utility and a cost to each node $v \in V$, where*

the utility and cost of v depend only on the label of v . Then, the local rounding framework can be adapted to support node utilities and costs.

Now, we present the local rounding lemma, which is used directly in the algorithms we introduce in Sections 3 and 4.

Lemma 2.4 (Lemma 2.5 of [FGG⁺23]). *Let $G = (V, E)$ be a graph equipped with utility and cost functions $\mathbf{u}$ and $\mathbf{c}$ and with a fractional label assignment λ such that for every label $\alpha \in \Sigma$ and every $v \in V$, if $\lambda_\alpha(v) > 0$, then $\lambda_\alpha(v) \geq \lambda_{\min}$ for some given value $\lambda_{\min} \in (0, 1]$. Further assume that G is equipped with a proper ζ -vertex coloring. Let $\delta \in [0, 1]$ and $\mu \in (0, 1]$ be two parameters. If $\mathbf{u}(\lambda) - \mathbf{c}(\lambda) \geq \mu \mathbf{u}(\lambda)$ and if each node knows the utility and cost functions of its incident edges, there is a deterministic $O\left(\frac{1}{\delta\mu} \cdot \log^2\left(\frac{1}{\delta\mu\lambda_{\min}}\right) + \log\left(\frac{1}{\delta\mu\lambda_{\min}}\right) \cdot \log^*\zeta\right)$ -round distributed message passing algorithm on G that computes an integral label assignment ℓ for which*

$$\mathbf{u}(\ell) - \mathbf{c}(\ell) \geq (1 - \delta) \cdot (\mathbf{u}(\lambda) - \mathbf{c}(\lambda)).$$

2.3 Clustering

In our weighted set cover algorithm given in Section 4, we use a relaxed variant of network decomposition. Ghaffari and Grunau [GG23] proposed a clustering algorithm where the clusters can have a lower diameter than $\Theta(\log n)$. This relaxation replaces the strict non-adjacency requirement within blocks with a bound on the number of neighboring clusters per node.

Theorem 2.5 (Theorem 2.1 of [GG23]). *Let $\alpha \leq \log n$ be an arbitrary value. There exists a deterministic distributed algorithm that in $\tilde{O}(\alpha^2 \log n)$ rounds of the LOCAL model computes a partition C with*

1. *diameter $O(\alpha)$ and*
2. *$\max_{v \in V} \deg_C(v) = 2^{\tilde{O}(\log n)/\alpha}$, i.e., each node has neighbors in at most $2^{\tilde{O}(\log n)/\alpha}$ clusters.*

3 Unweighted Set Cover

For the unweighted case of the distributed set cover problem, Faour et al. [FGG⁺23] presented a deterministic algorithm with $O(\log s)$ -approximation ratio. However, the hidden constants in the term $O(\log s)$ may be large. Our aim is to design a deterministic algorithm with an approximation ratio approaching $2\lceil \ln s \rceil$. Therefore, this section is dedicated to proving the following theorem:

Theorem 1.1. *There is a deterministic distributed algorithm in the LOCAL model that computes a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate set cover in $O(\log s \cdot \log^2 t + \log^* n) \cdot \text{poly}(1/\epsilon)$ rounds, where each set has at most s elements ($s \geq \max_{v \in V} \deg(v)$), each element is contained in at most t sets ($t \geq \max_{u \in U} \deg(u)$), and $0 < \epsilon \leq 1$.*

The $O(\log s)$ -approximation algorithm from [FGG⁺23] is a derandomization of a randomized set cover algorithm. We give a small description and a discussion of this randomized algorithm to provide insight into the intuition behind the deterministic algorithms presented subsequently in this and the next section.

Algorithm 1 Randomized Set Cover

Input: A bipartite graph $G = (U \sqcup V, E)$

Output: A set $V_{out} \subseteq V$ with $N(V_{out}) = U$

```

1:  $\tau \leftarrow \lceil \ln s \rceil$ 
2:  $\vec{x} : V \rightarrow [0, 1]$  is a 2-approximate fractional set cover of [KMW06]
3: for  $i \leftarrow 0, \dots, \tau - 1$  do
4:    $V_{i+1} \leftarrow \emptyset$ 
5:   Each  $v \in V$  joins  $V_{i+1}$  with probability  $x_v$ 
6: end for
7:  $U_\tau \leftarrow U \setminus N(V_1 \cup V_2 \cup \dots \cup V_\tau)$ 
8: Define  $V_{rem}$  by letting each element  $u \in U_\tau$  choose an arbitrary neighbor
9: return  $V_{out} = V_1 \cup \dots \cup V_\tau \cup V_{rem}$ 

```

We run the 2-approximate fractional set cover algorithm of Kuhn, Moscibroda, and Wattenhofer [KMW06] on the input graph G . Here, we obtain a function $x : V \rightarrow [0, 1]$ where x_v represents the fractional value assigned to the set $v \in V$ such that all elements $u \in U$ are covered, i.e., for all $u \in U$, $\sum_{v \in N(u)} x_v \geq 1$. Then, in each round $i = 0, \dots, \tau - 1$, we add each $v \in V$ independently to V_{i+1} with probability x_v . After τ rounds, $V_1 \cup \dots \cup V_\tau$ is the set of selected nodes for the covering set. We finish the algorithm by adding an arbitrary neighboring node $v \in V$ to the cover for each remaining uncovered element $u \in U_\tau$. The randomized algorithm has the expected approximation ratio of $(1 + 2\lceil \ln s \rceil)$.

As a derandomization method for this randomized algorithm, one can simulate each round of probabilistic

selection of the nodes $v \in V$ (line 5) with local rounding [FGG⁺23]. Two factors must be considered when nodes $v \in V$ are selected each round. First, the number of covered elements $u \in U$ from the remaining uncovered set of elements should be maximized. For instance, in round i we should cover as many nodes as possible from the set U_i , where U_i is the set of uncovered elements until round i . Second, the number of selected nodes $v \in V$ should be minimized since we wish to obtain a small covering set. These two conflicting optimizations are represented in the utility, $\mathbf{u}$, and the cost, $\mathbf{c}$, functions of the local rounding framework. One interesting aspect of having two optimizations is that their relative weight might change for the duration of τ rounds. Since the cardinality of the sets U_i decreases when i is iterated from 0 to $\tau - 1$, the relative weight of the first optimization should be increased. This can be achieved by multiplying the terms representing the first optimization in $\mathbf{u}$ and $\mathbf{c}$ by $1/a^{\tau-i}$; this causes the increase in the weight of the corresponding terms over rounds $i = 0, \dots, \tau - 1$. We call the constant a the adjustment constant. All deterministic algorithms presented in this thesis derandomize the given randomized algorithm using local rounding. The selected utility and cost functions are similar, and all originate from the two optimizations we have introduced above. Details of the selections are given in the respective subsections of this and the next section.

The $O(\log s)$ -approximation algorithm from [FGG⁺23] is given in Algorithm 2 with minor changes for clarity.

Algorithm 2 Deterministic $O(\log s)$ -Approximate Set Cover (Taken from [FGG⁺23])

Input: A bipartite graph $G = (U \sqcup V, E)$
Output: A set $V_{out} \subseteq V$ with $N(V_{out}) = U$

- 1: $\tau \leftarrow O(\log s)$
- 2: $\vec{x}_0 : V \rightarrow [0, 1]$ is a 2-approximate fractional set cover of [KMW06]
- 3: For every $v \in V$, define $x_v \leftarrow x_0(v)/10$ unless $x_0(v) \leq 1/(2t)$ in which case $x_v \leftarrow 0$.
- 4: For every $u \in U$, define $N^*(u) \subseteq N(u)$ such that $\frac{1}{20} \leq \sum_{v \in N^*(u)} x_v \leq \frac{1}{5}$
- 5: Compute 2-hop coloring of G with $\zeta = (st)^2$ colors using Linial's algorithm [Lin87]
- 6: $U_0 \leftarrow U$
- 7: **for** $i \leftarrow 0, \dots, \tau - 1$ **do**
- 8: $\mathbf{u}(\vec{x}) \leftarrow \frac{1}{1.01^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} \tilde{x}_v + 10 \sum_{v \in V} x_v$
- 9: $\mathbf{c}(\vec{x}) \leftarrow \frac{1}{1.01^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} \tilde{x}_v \tilde{x}_{v'} + \sum_{v \in V} \tilde{x}_v$
- 10: Plug in $\vec{x} = \vec{x}$, use Lemma 2.4 (Lemma 2.5 of [FGG⁺23]) with $\mathbf{u}, \mathbf{c}, \zeta, \delta = \frac{1}{100}, \mu = \frac{1}{2}, \lambda_{\min} = \frac{1}{20t}$,
to round $\vec{x}$ to $\vec{x}'_i$
- 11: $V_{i+1} \leftarrow \{v \in V \mid x'_i(v) = 1\}$
- 12: $U_{i+1} \leftarrow U \setminus N(V_1 \cup V_2 \dots \cup V_{i+1})$
- 13: **end for**
- 14: Define V_{rem} by letting each element $u \in U_\tau$ choose an arbitrary neighbor
- 15: **return** $V_{out} = V_1 \cup \dots \cup V_\tau \cup V_{rem}$

With some alterations to this algorithm, we construct an approximation scheme and prove Theorem 1.1. The approximation scheme is given in Algorithm 3.

The rest of the section is devoted to the description and analysis of Algorithm 3. First, we provide a high-level overview of the algorithm. As discussed previously, a bipartite graph $G = (U \sqcup V, E)$ is given as an input, along with a parameter ϵ for the approximation ratio. This parameter is restricted to $0 < \epsilon \leq 1$. To start off, we run the algorithm of Kuhn, Moscibroda, Wattenhofer [KMW06] to obtain a 2-approximate

Algorithm 3 Deterministic $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -Approximate Set Cover

Input: A bipartite graph $G = (U \sqcup V, E)$, parameter $\epsilon \in (0, 1]$

Output: A set $V_{out} \subseteq V$ with $N(V_{out}) = U$

- 1: $\vec{x}_0 : V \rightarrow [0, 1]$ is a 2-approximate fractional set cover of [KMW06]
 - 2: For every $v \in V$, define $x_v \leftarrow x_0(v)\epsilon/10$ unless $x_0(v) \leq \epsilon/(50t)$ in which case $x_v \leftarrow 0$.
 - 3: For every $u \in U$, define $N^*(u) \subseteq N(u)$ such that $\frac{(50-\epsilon)\epsilon}{500} \leq \sum_{v \in N^*(u)} x_v \leq \frac{(100-\epsilon)\epsilon}{500}$
 - 4: $d \leftarrow 1 + (1 - \frac{\epsilon}{20}) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250} \right)$
 - 5: $\tau \leftarrow \lceil 1/\ln d \rceil \lceil \ln s \rceil$
 - 6: Compute 2-hop coloring of G with $\zeta = (st)^2$ colors using Linial's algorithm [Lin87]
 - 7: $U_0 \leftarrow U$
 - 8: **for** $i \leftarrow 0, \dots, \tau - 1$ **do**
 - 9: $\mathbf{u}(\vec{x}) \leftarrow \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} \tilde{x}_v + 2 \sum_{v \in V} x_v$
 - 10: $\mathbf{c}(\vec{x}) \leftarrow \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} \frac{1}{2} \sum_{v' \neq v \in N^*(u)} \tilde{x}_v \tilde{x}_{v'} + \sum_{v \in V} \tilde{x}_v$
 - 11: Plug in $\vec{x} = \vec{x}$, use Lemma 2.4 (Lemma 2.5 of [FGG⁺23]) with $\mathbf{u}, \mathbf{c}, \zeta, \delta = \frac{\epsilon}{20}, \mu = \frac{1}{2}, \lambda_{\min} = \frac{\epsilon^2}{500t}$,
to round $\vec{x}$ to $\vec{x}'_i$
 - 12: $V_{i+1} \leftarrow \{v \in V \mid x'_i(v) = 1\}$
 - 13: $U_{i+1} \leftarrow U \setminus N(V_1 \cup V_2 \dots \cup V_{i+1})$
 - 14: **end for**
 - 15: Define V_{rem} by letting each element $u \in U_\tau$ choose an arbitrary neighbor
 - 16: **return** $V_{out} = V_1 \cup \dots \cup V_\tau \cup V_{rem}$
-

fractional set cover solution on G . We convert this 2-approximate solution to an $O(\epsilon^2/t)$ -integral solution. Then, we apply local rounding to the new fractional solution for $\tau = O(\ln s / \ln(1 + \epsilon))$ rounds. In each round, the $O(\epsilon^2/t)$ -integral fractional solution is rounded up to a binary solution, i.e., each node $v \in V$ stores either 1 or 0 in its local variable. If a node $v \in V$ stores 1 in any of the τ rounds, then it is included in the covering set. However, after τ rounds there might be some elements $u \in U$ that are not yet covered. For each uncovered $u \in U$, we add an arbitrary neighbor of u to the covering set. After the last step, we have covered all the elements $u \in U$; therefore, we can terminate. We describe the algorithm in more detail in the next subsections.

3.1 Obtaining Fractional Solutions

The first three lines of the algorithm calculate a 2-approximate fractional solution to the given set cover instance, and then use this solution to obtain a $1/K$ -integral fractional solution. In line 1, using the algorithm from [KMW06], each $v \in V$ stores a value $x_0(v)$ where the collection of all $x_0(v)$ establishes a 2-approximate fractional solution. We represent this collection of values as a vector. Considering the values $\vec{x}_0$, a value $x_0(v)$ can be arbitrarily close to 0. This is a problem, as the subsequent local rounding phase has a round complexity dependent on the minimum value to be rounded. In Lemma 2.4, we can observe this fact by spotting the variable $\lambda_{\min}$ in the round complexity. This is the motivation behind the creation of a second solution, a $1/K$ -integral solution, in which we cannot have values between 0 and $1/K$.

In line 2, we construct an $(\epsilon^2/500t)$ -integral solution, $\vec{x}$. For all $v \in V$, x_v is set to 0 if $x_0(v)$ is sufficiently small; otherwise it is set to $x_0(v) \cdot (\epsilon/10)$. Then, in line 3, for all $u \in U$, a subset $N^*(u)$ is created from the neighbors of u , namely $N(u)$. Each subset is used to select nodes $v \in N^*(u)$ in order to cover

u later. However, the nodes $v \in N^*(u)$ must have small x_v values; this ensures that the approximate solution returned by Algorithm 3 has a cardinality close to the optimal solution. Now, we demonstrate in the following two claims that for each element $u \in U$ we can find such a subset $N^*(u)$.

First, we establish that after generating the $1/K$ -integral solution, every $u \in U$ is fractionally covered by at least a certain amount.

Claim 3.1. *For every $u \in U$,*

$$\sum_{v \in N(u)} x_v \geq \frac{(50 - \epsilon)\epsilon}{500}$$

Proof. Let $N_s(u) \subseteq N(u)$ be the set of nodes v that have small values from the 2-approximate fractional solution, that is, $x_0(v) \leq \epsilon/50t$. Since $|N_s(u)| \leq |N(u)| \leq t$:

$$\sum_{v \in N_s(u)} x_0(v) \leq t \cdot \frac{\epsilon}{50t} = \frac{\epsilon}{50}$$

The 2-approximate solution covers every element $u \in U$ ($\sum_{v \in N(u)} x_0(v) \geq 1$). Then, the contribution of non-small nodes $v \in N(u) \setminus N_s(u)$ to the covering of u should be at least $1 - (\epsilon/50)$. Hence,

$$\sum_{v \in N(u)} x_v = \sum_{v \in N(u) \setminus N_s(u)} x_v \geq \left(1 - \frac{\epsilon}{50}\right) \cdot \frac{\epsilon}{10} = \frac{(50 - \epsilon)\epsilon}{500}$$

The first equality is derived from the fact that in line 2, we assign the values of nodes $v \in V$ to 0 if $x_0(v) \leq \epsilon/50t$. $\square$

Now, we can show that $N^*(u)$ exists for all $u \in U$ satisfying the lower and upper bound given in line 3:

Claim 3.2. *There exists an $N^*(u) \subseteq N(u)$ for all $u \in U$, such that*

$$\frac{(50 - \epsilon)\epsilon}{500} \leq \sum_{v \in N^*(u)} x_v \leq \frac{(100 - \epsilon)\epsilon}{500}$$

Proof. We create the set $N^*(u)$ by adding nodes from $N(u)$ for each $u \in U$. From Claim 3.1, we can always add enough nodes from $N(u)$ such that $\sum_{v \in N^*(u)} x_v \geq \frac{(50 - \epsilon)\epsilon}{500}$. We observe that the greatest value any $v \in V$ can take in the $1/K$ -integral solution is $\epsilon/10$. Thus,

$$\begin{aligned} \frac{(50 - \epsilon)\epsilon}{500} &\leq \sum_{v \in N^*(u)} x_v \leq \frac{\epsilon}{10} + \frac{(50 - \epsilon)\epsilon}{500} \\ &= \frac{(100 - \epsilon)\epsilon}{500} \end{aligned}$$

$\square$

Lastly, we need to establish an upper bound for the sum of the fractional values of the $1/K$ -integral solution. We use this bound later in the analysis of the algorithm.

Claim 3.3.

$$\sum_{v \in V} x_v \leq \frac{\epsilon}{10} \cdot 2 \cdot OPT$$

Proof. $\vec{x}_0$ is a 2-approximate fractional solution. Therefore,

$$\sum_{v \in V} x_0(v) \leq 2 \cdot OPT$$

Then, in the $1/K$ -integral solution, either a node $v \in V$ stores an $(\epsilon/10)$ fraction of its initial value, or 0. $\square$

3.2 Rounding $1/K$ -Integral Solution

The remaining steps of Algorithm 3 (lines 4–16) implement the local rounding procedure. We have already provided a short description of local rounding in Section 1.1 and defined the constructs of the framework in Section 2.2. Furthermore, we have mentioned at the beginning of this section that local rounding is used as a derandomization technique: to create deterministic set cover algorithms from the randomized algorithm (Algorithm 1). In this part of the algorithm, we aim to generate τ binary solutions $\vec{x}'$ by rounding the $(1/K)$ -integral solution $\vec{x}$. Each time we round $\vec{x}$ to $\vec{x}'$, we add $v \in V$ to the covering set if $x'_v = 1$.

In lines 4 and 5 we assign values to d , the adjustment constant, and τ , the number of times we apply the rounding subroutine, respectively. The selected values for these constants become clear towards the end of this section and in the beginning of Section 3.3 where we analyze the approximation ratio and round complexity.

Before applying the local rounding lemma (Lemma 2.4), we need to color the graph using Linial's algorithm [Lin87]. This coloring is a requirement for the local rounding framework, also mentioned in Lemma 2.4. It is important to note that we use 2-hop coloring instead of a normal coloring. The cost function, which will be discussed shortly, uses a sum of terms $x_v x_{v'}$ for some $v, v' \in N(u)$ ($v \neq v'$). However, there is no edge $\{v, v'\}$ in G . Therefore, local rounding simulates the communication between v and v' by using two edges, through a common neighbor $u \in U$.

Utility and cost functions are defined for every possible fractional solution, according to Definition 2.6. Although we only round the specific $1/K$ -integral solution $\vec{x}$ we obtained in line 2, we must define $\mathbf{u}$ and $\mathbf{c}$ for every possible fractional solution. We denote the arbitrary fractional solution with $\vec{x}$. For selecting the utility and the cost functions, we need to consider our optimization goals. First, we want to maximize the elements $u \in U$ we cover in each round. We want to round a fractional solution $\vec{x}$ to maximize the following expression, a pessimistic lower bound for the expected value of $|U_i \setminus U_{i+1}|$ in the randomized version of the algorithm ([FGG⁺23]):

$$\sum_{u \in U_i} \left(\sum_{v \in N(u)} \tilde{x}_v - \sum_{v \in N(u)} \sum_{v' \neq v \in N(u)} \tilde{x}_v \tilde{x}_{v'} \right) \quad (1)$$

This estimator evaluates to a value less than or equal to 1 for each $u \in U_i$. If we interpret all $\tilde{x}_v$ as probabilities for selection, the inner part of the expression (inside the parentheses) is a lower bound for the probability that at least one node $v \in N(u)$ is selected (which makes u covered), for each $u \in U_i$. To understand how the expression evaluates for an arbitrary $u \in U_i$, we demonstrate all possibilities below: Suppose we have a fractional solution $\vec{x}$. Then, for some $u \in U_i$:

- If $\tilde{x}_v = 1$ for only one $v \in N(u)$ and $\tilde{x}_{v'} = 0$ for other $v' \in N(u)$, evaluates to 1
- If $\sum_{v \in N(u)} \tilde{x}_v \geq 1$ and more than one $v \in N(u)$ have $\tilde{x}_v > 0$, evaluates to a value less than 1
- If $0 < \sum_{v \in N(u)} \tilde{x}_v < 1$, evaluates to a value less than or equal to $\sum_{v \in N(u)} \tilde{x}_v$
- If $\tilde{x}_v = 0$ for all $v \in N(u)$, evaluates to 0

However, this expression does not yield a tight lower bound for all fractional solutions. For example, if $\tilde{x}_v = 1$ for all $v \in N(u)$, the expression evaluates to an unbounded negative value depending on the size of $N(u)$ (precisely $2|N(u)| - |N(u)|^2$), whereas we expect to get a value close to 1. Our proposed expression for this optimization is given below:

$$\sum_{u \in U_i} \left(\sum_{v \in N(u)} \tilde{x}_v - \frac{1}{2} \sum_{v \in N(u)} \sum_{v' \neq v \in N(u)} \tilde{x}_v \tilde{x}_{v'} \right) \quad (2)$$

This estimator is asymptotically the same as the previous one, although it allows us to tighten our analysis and achieve the desired approximation ratio. We show that the new expression gives a lower bound for the expected value of $|U_i \setminus U_{i+1}|$ for the randomized algorithm. Let $\vec{x}$ be a fractional solution and let $u \in U$. Then,

- If $\sum_{v \in N(u)} \tilde{x}_v \leq 1$, then the inner expression evaluates to a value less than or equal to $\sum_{v \in N(u)} \tilde{x}_v$
- If $\sum_{v \in N(u)} \tilde{x}_v > 1$, then the inner expression returns a value less than or equal to 1. This is not obvious; therefore, we provide a small proof below: We apply induction on the number of neighbors u has. Since $\sum_{v \in N(u)} \tilde{x}_v > 1$, there must be at least two $v \in N(u)$ such that $\tilde{x}_v > 0$. For the base case, let $N(u) = \{v_1, v_2\}$. Clearly, $\tilde{x}_{v_1} + \tilde{x}_{v_2} - \tilde{x}_{v_1} \tilde{x}_{v_2} \leq 1$ for all $0 \leq \tilde{x}_{v_1}, \tilde{x}_{v_2} \leq 1$. Now, let u have k neighbors, namely $v_1, \dots, v_k \in N(u)$ and assume

$$\sum_{i=1}^k \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^k \sum_{\substack{j=1 \\ j \neq i}}^k \tilde{x}_{v_i} \tilde{x}_{v_j} \leq 1$$

for all $0 \leq \tilde{x}_{v_1}, \dots, \tilde{x}_{v_k} \leq 1$. For $N(u) = \{v_1, \dots, v_{k+1}\}$ we need to prove the inequality below:

$$\sum_{i=1}^{k+1} \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^{k+1} \sum_{\substack{j=1 \\ j \neq i}}^{k+1} \tilde{x}_{v_i} \tilde{x}_{v_j} \leq 1$$

for all $0 \leq \tilde{x}_{v_1}, \dots, \tilde{x}_{v_{k+1}} \leq 1$. Rewriting the expression:

$$\sum_{i=1}^{k+1} \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^{k+1} \sum_{\substack{j=1 \\ j \neq i}}^{k+1} \tilde{x}_{v_i} \tilde{x}_{v_j} = \sum_{i=1}^k \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^k \sum_{\substack{j=1 \\ j \neq i}}^k \tilde{x}_{v_i} \tilde{x}_{v_j} + \tilde{x}_{v_{k+1}} - \frac{1}{2} \left(\sum_{i=1}^k \tilde{x}_{v_{k+1}} \tilde{x}_{v_i} + \sum_{i=1}^k \tilde{x}_{v_i} \tilde{x}_{v_{k+1}} \right)$$

From the assumption, there exists a c such that

$$\sum_{i=1}^k \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^k \sum_{\substack{j=1 \\ j \neq i}}^k \tilde{x}_{v_i} \tilde{x}_{v_j} = c \leq 1$$

Then,

$$\begin{aligned} \sum_{i=1}^{k+1} \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^{k+1} \sum_{\substack{j=1 \\ j \neq i}}^{k+1} \tilde{x}_{v_i} \tilde{x}_{v_j} &= \sum_{i=1}^k \tilde{x}_{v_i} - \frac{1}{2} \sum_{i=1}^k \sum_{\substack{j=1 \\ j \neq i}}^k \tilde{x}_{v_i} \tilde{x}_{v_j} + \tilde{x}_{v_{k+1}} - \frac{1}{2} \left(\sum_{i=1}^k \tilde{x}_{v_{k+1}} \tilde{x}_{v_i} + \sum_{i=1}^k \tilde{x}_{v_i} \tilde{x}_{v_{k+1}} \right) \\ &= c + \tilde{x}_{v_{k+1}} - \frac{1}{2} \left(\sum_{i=1}^k \tilde{x}_{v_{k+1}} \tilde{x}_{v_i} + \sum_{i=1}^k \tilde{x}_{v_i} \tilde{x}_{v_{k+1}} \right) \\ &= c + \tilde{x}_{v_{k+1}} - \sum_{i=1}^k \tilde{x}_{v_i} \tilde{x}_{v_{k+1}} \\ &\leq c + \tilde{x}_{v_{k+1}} - c \tilde{x}_{v_{k+1}} \\ &= c(1 - \tilde{x}_{v_{k+1}}) + \tilde{x}_{v_{k+1}} \\ &\leq 1 \end{aligned}$$

The last inequality holds since $0 \leq \tilde{x}_{v_{k+1}} \leq 1$ and $c \leq 1$.

The second optimization, the minimization of the selected nodes $v \in V$ in each round, can be represented rather easily with the sole term $\sum_{v \in V} \tilde{x}_v$. Since we obtain mathematical expressions for both of the optimizations, we can now define the utility and cost functions.

$$\begin{aligned} \mathbf{u}(\vec{x}) &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} \tilde{x}_v + 2 \sum_{v \in V} x_v \\ \mathbf{c}(\vec{x}) &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} \tilde{x}_v \tilde{x}_{v'} + \sum_{v \in V} \tilde{x}_v \end{aligned}$$

Here, we distributed the terms $\sum_{u \in U_i} \sum_{v \in N(u)} \tilde{x}_v$ and $\sum_{u \in U_i} 1/2 \sum_{v \in N(u)} \sum_{v' \neq v \in N(u)} \tilde{x}_v \tilde{x}_{v'}$ from the first optimization to the utility and cost, respectively. Concerning the second optimization, inserting $\sum_{v \in V} \tilde{x}_v$ into the cost function makes sense, as we need to minimize the term. We added the adjustment constant d in front of the relevant terms. We have already justified the need for an adjustment constant in the beginning of the current section. Moreover, we added the term $2 \sum_{v \in V} x_v$ to the utility function as a constant offset based on the $1/K$ -integral fractional solution, in order to conform to the requirement of Lemma 2.4. Since each rounding step is applied to the $1/K$ -integral solution $\vec{x}$, we need only consider the expressions $\mathbf{u}(\vec{x})$ and $\mathbf{c}(\vec{x})$, rather than their generalized counterparts $\mathbf{u}(\tilde{x})$ and $\mathbf{c}(\tilde{x})$. Before delving into analyzing each round of local rounding, we need to establish that the selected utility and cost functions

are proper according to Lemma 2.4 for the vector $\vec{x}$:

Claim 3.4. $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) \geq \mathbf{u}(\vec{x})/2$

Proof. We prove the equivalent statement $\mathbf{c}(\vec{x}) \leq \mathbf{u}(\vec{x})/2$ instead. We compare the first and second terms of $\mathbf{u}(\vec{x})$ and $\mathbf{c}(\vec{x})$ separately. Comparing the first terms:

$$\begin{aligned}
\frac{1}{d^{\tau-i}} \sum_{u \in U_i} \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} &\leq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \frac{1}{2} \sum_{v \in N^*(u)} \left(x_v \cdot \left(\sum_{v' \in N^*(u)} x_{v'} \right) \right) \\
&\leq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \frac{1}{2} \sum_{v \in N^*(u)} \left(x_v \cdot \frac{(100-\epsilon)\epsilon}{500} \right) \\
&= \frac{(100-\epsilon)\epsilon}{1000} \cdot \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} x_v \\
&< \frac{1}{2} \left(\frac{1}{d^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} x_v \right)
\end{aligned}$$

For the second terms, we can observe that $\sum_{v \in V} x_v = \frac{1}{2}(2 \sum_{v \in V} x_v)$. Here, one can see that the term $2 \sum_{v \in V} x_v$ is added to $\mathbf{u}$ for this purpose. $\square$

One intermediate inequality from the proof of Claim 3.4 is especially important for the analysis of the algorithm. We restate the inequality below:

$$\frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} \leq \frac{(100-\epsilon)\epsilon}{1000} \sum_{v \in N^*(u)} x_v \quad (3)$$

3.2.1 Analyzing One Round of Local Rounding

We define an indicator variable, $R_u(\vec{x}')$: It indicates whether an element $u \in U$ is covered ($R_u(\vec{x}') = 1$) or not covered ($R_u(\vec{x}') = 0$) for a binary solution $\vec{x}'$. Then, for any binary solution $\vec{x}'$ and for any $u \in U$,

$$R_u(\vec{x}') \geq \sum_{v \in N^*(u)} \left(\tilde{x}'_v - \frac{1}{2} \sum_{v' \neq v \in N^*(u)} \tilde{x}'_v \tilde{x}'_{v'} \right)$$

For $R_u(\vec{x}') = 0$, it is easy to see the inequality holds: For all $v \in N(u)$, $\tilde{x}'_v$ must be 0. We discussed in the last section that the expression $\left(\sum_{v \in N(u)} \tilde{x}_v - \frac{1}{2} \sum_{v \in N(u)} \sum_{v' \neq v \in N(u)} \tilde{x}_v \tilde{x}_{v'} \right)$ cannot be greater than 1 for any fractional solution $\vec{x}$. Since any binary solution is also a fractional solution, the inequality also holds in the cases where $R_u(\vec{x}') = 1$.

We round the $1/K$ -integral fractional solution $\vec{x}$ to a binary solution $\vec{x}'$. For the rounded values $\vec{x}'$, using the definitions of $\mathbf{u}$ and $\mathbf{c}$ we can derive

$$\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}') - 2 \sum_{v \in V} x_v \leq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} R_u(\vec{x}') - \sum_{v \in V} x'_v = \frac{1}{d^{\tau-i}} (|U_i| - |U_{i+1}|) - |V_{i+1}| \quad (4)$$

Here, we upper bounded the expression $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}')$. Next, our aim is to find a lower bound for the same expression. We accomplish this by inspecting the expression $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})$ with $1/K$ -integral fractional values first. Then, according to Lemma 2.4, $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}')$ is bounded by a $(1 - \delta)$ factor of $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})$. Looking at $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})$, we have

$$\begin{aligned}
\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} x_v &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \sum_{v \in N^*(u)} \left(x_v - \frac{1}{2} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} \right) - \sum_{v \in V} x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(\sum_{v \in N^*(u)} x_v - \frac{(100 - \epsilon)\epsilon}{1000} \sum_{v \in N^*(u)} x_v \right) - \sum_{v \in V} x_v \\
&= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \sum_{v \in N^*(u)} x_v - \sum_{v \in V} x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \left(\frac{(50 - \epsilon)\epsilon}{500} \right) - \sum_{v \in V} x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \left(\frac{(50 - \epsilon)\epsilon}{500} \right) - \frac{2\epsilon}{10} \cdot OPT \\
&= \frac{1}{d^{\tau-i}} \cdot |U_i| \left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \left(\frac{(50 - \epsilon)\epsilon}{500} \right) - \frac{2\epsilon}{10} \cdot OPT
\end{aligned}$$

The first equality follows from the definitions of $\mathbf{u}(\vec{x})$ and $\mathbf{c}(\vec{x})$. In the first inequality, we replace the quadratic term, $1/2 \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} x_v x_{v'}$, by using Eq. (3). In the second inequality, we use Claim 3.2 to substitute the term $\sum_{v \in N^*(u)} x_v$ with its lower bound. We replace the term $\sum_{v \in V} x_v$ with its upper bound given in Claim 3.3, in the last inequality.

Working with the expression $\left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \left(\frac{(50 - \epsilon)\epsilon}{500} \right)$ leads to cumbersome terms in equations. Thus, we introduce a small claim which allows us to use a simpler expression instead.

Claim 3.5. For $0 < \epsilon \leq 1$,

$$\left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \left(\frac{(50 - \epsilon)\epsilon}{500} \right) \geq \frac{\epsilon}{10} - \frac{3\epsilon^2}{250}$$

Proof.

$$\begin{aligned}
\left(1 - \frac{(100 - \epsilon)\epsilon}{1000} \right) \left(\frac{(50 - \epsilon)\epsilon}{500} \right) &= \frac{\epsilon}{10} - \frac{3\epsilon^2}{250} + \frac{3\epsilon^3}{10000} - \frac{\epsilon^4}{500000} \\
&\geq \frac{\epsilon}{10} - \frac{3\epsilon^2}{250}
\end{aligned}$$

Clearly $3\epsilon^3/10^4 \geq \epsilon^4/(5 \cdot 10^5)$ when $0 < \epsilon \leq 1$, hence the inequality holds. $\square$

If we rewrite the lower bound for $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})$ with the simpler expression, we get

$$\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} x_v \geq \frac{1}{d^{\tau-i}} \cdot |U_i| \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250} \right) - \frac{2\epsilon}{10} \cdot OPT \quad (5)$$

We relate $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})$, evaluated at the fractional solution $\vec{x}$, to $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}')$, evaluated at the rounded values $\vec{x}'$, using Lemma 2.4 with $\delta = \epsilon/20$.

$$\begin{aligned}
\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}') - 2 \sum_{v \in V} x_v &\geq \left(1 - \frac{\epsilon}{20}\right) \cdot (\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})) - 2 \sum_{v \in V} x_v \\
&= \left(1 - \frac{\epsilon}{20}\right) \cdot \left(\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} x_v\right) - \frac{2\epsilon}{20} \sum_{v \in V} x_v \\
&\geq \left(1 - \frac{\epsilon}{20}\right) \cdot \left(\frac{1}{d^{\tau-i}} \cdot |U_i| \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right) - \frac{2\epsilon}{10} \cdot OPT\right) - \frac{2\epsilon}{20} \sum_{v \in V} x_v \\
&\geq \left(1 - \frac{\epsilon}{20}\right) \cdot \left(\frac{1}{d^{\tau-i}} \cdot |U_i| \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right) - \frac{2\epsilon}{10} \cdot OPT\right) - \frac{2\epsilon}{20} \frac{2\epsilon}{10} \cdot OPT \\
&= \frac{1}{d^{\tau-i}} \cdot |U_i| \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right) - \left(1 + \frac{\epsilon}{20}\right) \frac{2\epsilon}{10} \cdot OPT
\end{aligned} \tag{6}$$

In the second inequality, we substitute the term $(\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} x_v)$ using Eq. (5). Claim 3.3 is used for the third inequality.

We established an upper bound in Eq. (4) and a lower bound in Eq. (6) for the term $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}')$. Putting these two bounds together, we derive the following:

$$\frac{1}{d^{\tau-i}} (|U_i| - |U_{i+1}|) - |V_{i+1}| \geq \frac{1}{d^{\tau-i}} \cdot |U_i| \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right) - \left(1 + \frac{\epsilon}{20}\right) \frac{2\epsilon}{10} \cdot OPT$$

Selecting $d = 1 + \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)$ gives us:

$$\frac{1}{d^{\tau-i}} (|U_i| - |U_{i+1}|) - |V_{i+1}| \geq \frac{1}{d^{\tau-i}} \cdot |U_i| \cdot (d - 1) - \left(1 + \frac{\epsilon}{20}\right) \frac{2\epsilon}{10} \cdot OPT \tag{7}$$

This result can be interpreted as follows: Every round, we are roughly covering $(d - 1)$ fraction of the uncovered U_i where $d - 1$ is a positive number smaller than 1. In order to achieve this goal, we select a set $V_{i+1} \subseteq V$ such that the number of selected nodes $|V_{i+1}|$ is approximately less than the sum of fractional x_v values of all $v \in V$ multiplied by $\left(1 + \frac{\epsilon}{20}\right)$. Eq. (7) plays a crucial role in the analysis of the approximation ratio, which we explain further in the next subsection.

3.3 Approximation Ratio and Round Complexity

In this section, we finish proving Theorem 1.1. First, we show that the algorithm returns a solution with the approximation ratio $(1 + (2 + \epsilon)\lceil \ln s \rceil)$. Then, we discuss the round complexity.

We define a potential function for every step of the rounding.

Definition 3.1. For all $0 \leq i \leq \tau$,

$$\Phi_i = \frac{1}{d^{\tau-i}} \cdot |U_i| + |V_1| + \dots + |V_i| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) (\tau - i) \cdot OPT$$

In this definition, Φ_0 does not correspond to a rounding step; it represents the state that exists just before the local rounding procedure starts (before line 8). Φ_τ also has its own importance; it corresponds to the end of the rounding procedure loop (line 14).

Our proof structure for establishing the approximation ratio is given below:

1. Prove that Φ_0 is less than or equal to $(1 + (2 + \epsilon)\lceil \ln s \rceil) \cdot OPT$
2. Show $|V_{out}|$, the cardinality of the covering set the algorithm returns, is less than or equal to Φ_τ
3. Prove that the potential function Φ_i is monotonically non-increasing, thus $\Phi_\tau \leq \Phi_0$.

From these steps, we can infer that $|V_{out}| \leq (1 + (2 + \epsilon)\lceil \ln s \rceil) \cdot OPT$

We start off with the first task. We have,

$$\Phi_0 = \frac{1}{d^\tau} \cdot |U| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \tau \cdot OPT$$

Here, a suitable integer for τ is needed. We want to substitute the first term, $|U|/d^\tau$, with a simpler term $|U|/s$. However, the straightforward selection of $\tau = \lceil \log_d s \rceil$ makes the analysis cumbersome; working with a logarithmic term with base d may introduce terms that are hard to simplify. Instead, we select $\tau = \lceil 1/\ln d \rceil \lceil \ln s \rceil$. To relate these two selections:

$$\left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \geq \frac{\lceil \ln s \rceil}{\ln d} = \lceil \ln s \rceil \frac{\log_d s}{\ln s} \geq \lceil \ln s \rceil \frac{\log_d s}{\lceil \ln s \rceil} = \log_d s \quad (8)$$

The first equality follows from the logarithm laws for base change. If we rewrite Φ_0 with $\tau = \lceil 1/\ln d \rceil \lceil \ln s \rceil$:

$$\begin{aligned} \Phi_0 &= \frac{1}{d^{\lceil 1/\ln d \rceil \lceil \ln s \rceil}} \cdot |U| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\ &\leq \frac{1}{d^{\log_d s}} \cdot |U| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\ &= \frac{|U|}{s} + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \end{aligned}$$

In the inequality, we used Eq. (8). Next, we observe that $|U|/s \leq OPT$. This is easy to argue by contradiction: Suppose the optimal solution S contains fewer than $|U|/s$ sets. Then, since any set in S covers at most s elements, the solution covers at most $(|S| \cdot s)$ elements. $(|S| \cdot s)$ is less than $|U|$ which means there exists an element $u \in U$ which is not covered by the optimal solution S , a contradiction. Using this, we can further simplify the expression:

$$\begin{aligned} \Phi_0 &\leq \frac{|U|}{s} + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\ &\leq OPT + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\ &= \left(1 + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil\right) \cdot OPT \end{aligned}$$

We can plug in the value of d , after removing the ceiling from $\lceil 1/\ln d \rceil$ using $\lceil 1/\ln d \rceil \leq 1 + (1/\ln d)$:

$$\begin{aligned}
\Phi_0 &\leq \left(1 + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil\right) \cdot OPT \\
&\leq \left(1 + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left(1 + \frac{1}{\ln d}\right) \lceil \ln s \rceil\right) \cdot OPT \\
&= \left(1 + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left(1 + \frac{1}{\ln \left(1 + \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)\right)}\right) \lceil \ln s \rceil\right) \cdot OPT \\
&\leq \left(1 + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \left(1 + \frac{\left(1 + \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)\right)}{\left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)}\right) \lceil \ln s \rceil\right) \cdot OPT \\
&\stackrel{*}{\leq} \left(1 + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \frac{200 + 100\epsilon}{20\epsilon + \epsilon^2} \lceil \ln s \rceil\right) \cdot OPT \\
&= (1 + (2 + \epsilon) \lceil \ln s \rceil) \cdot OPT
\end{aligned}$$

In the fourth line, the following logarithmic inequality is used [Top07]: For all $\alpha > 0$,

$$\ln(\alpha) \geq 1 - \frac{1}{\alpha} = \frac{\alpha - 1}{\alpha}$$

Then, we further substitute a new value in the last inequality, $(200 + 100\epsilon)/(20\epsilon + \epsilon^2)$, in order to simplify the expression. Validity of this substitution (*) is trivial to verify; nevertheless, a brief proof is given in the appendix section.

To show $|V_{out}| \leq \Phi_\tau$, we give the definition of Φ_τ first.

$$\Phi_\tau = |U_\tau| + |V_1| + \dots + |V_\tau|$$

where U_τ is the set of elements $u \in U$ still uncovered after the local rounding procedure. In the algorithm, we cover every element in U_τ by selecting an arbitrary neighbor $v \in N(u)$ for every $u \in U_\tau$ (line 15) and adding these neighbors, namely V_{rem} , to the covering set. From this fact, we can conclude that $|V_{rem}| \leq |U_\tau|$. Then,

$$\begin{aligned}
|V_{out}| &= |V_1 \cup \dots \cup V_\tau \cup V_{rem}| \\
&\leq |V_1| + \dots + |V_\tau| + |V_{rem}| \\
&\leq |V_1| + \dots + |V_\tau| + |U_\tau| \\
&= \Phi_\tau
\end{aligned}$$

We prove the non-increasing monotonicity of the potential function Φ_i by showing $\Phi_i - \Phi_{i+1} \geq 0$ for an arbitrary $0 \leq i \leq \tau - 1$. Using the definition of the potential function:

$$\begin{aligned}
\Phi_i - \Phi_{i+1} &= \frac{1}{d^{\tau-i}} (|U_i| - d \cdot |U_{i+1}|) - |V_{i+1}| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \\
&= \frac{1}{d^{\tau-i}} (|U_i| - |U_{i+1}|) - |V_{i+1}| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT - (d - 1) \cdot \frac{|U_{i+1}|}{d^{\tau-i}}
\end{aligned}$$

Using Eq. (7) from the previous section:

$$\begin{aligned}
\Phi_i - \Phi_{i+1} &= \frac{1}{d^{\tau-i}}(|U_i| - |U_{i+1}|) - |V_{i+1}| + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT - (d-1) \cdot \frac{|U_{i+1}|}{d^{\tau-i}} \\
&\geq \frac{1}{d^{\tau-i}} \cdot |U_i| \cdot (d-1) - \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT + \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT - (d-1) \cdot \frac{|U_{i+1}|}{d^{\tau-i}} \\
&= \frac{1}{d^{\tau-i}}(d-1)(|U_i| - |U_{i+1}|) \\
&\geq 0
\end{aligned}$$

We have now proven that the algorithm returns a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate solution to the set cover problem. At last, we need to determine the round complexity of the algorithm. We analyze the subroutines, then sum them to find the total rounds. In line 1, we apply the 2-approximate fractional set cover algorithm from [KMW06]. This algorithm runs in $O(\log s \cdot \log t)$ rounds. Then, in line 6, we apply Linial's Δ^2 -coloring algorithm [Lin87] which has $O(\log^* n)$ round complexity. After the coloring, we utilize local rounding; we round the $1/K$ -fractional solution to a binary solution $\tau = \lceil 1/\ln d \rceil \lceil \ln s \rceil$ times. Each rounding step runs in

$$O\left(\frac{1}{\epsilon} \cdot \log^2\left(\frac{t}{\epsilon^3}\right) + \log\left(\frac{t}{\epsilon^3}\right) \cdot \log^*((st)^2)\right)$$

rounds, according to Lemma 2.4. Consequently, the whole local rounding procedure (lines 8-14) has

$$\left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot O\left(\frac{1}{\epsilon} \cdot \log^2\left(\frac{t}{\epsilon^3}\right) + \log\left(\frac{t}{\epsilon^3}\right) \cdot \log^*((st)^2)\right)$$

complexity. Clearly, $\lceil 1/\ln d \rceil = O\left(\frac{1}{\ln(1+\epsilon)}\right)$. We can further substitute $O\left(\frac{1}{\ln(1+\epsilon)}\right)$ with $O(1/\epsilon)$ as for all $0 < \epsilon \leq 1$,

$$\frac{1}{\ln(1+\epsilon)} \leq 2 \cdot \frac{1}{\epsilon}$$

Similarly, $O(1/\epsilon)$ dominates $O(\log^2(1/\epsilon^3))$. Also, $O(\log^*((st)^2)) = O(\log^*(st))$. After the simplifications, the round complexity of the rounding procedure can be represented as

$$O(\log s) \cdot O(\log^2 t + \log t \cdot \log^*(st)) \cdot \text{poly}(1/\epsilon)$$

In the term $O(\log^2 t + \log t \cdot \log^*(st))$, the first term dominates the second if $t > \ln s$. If we assume $t \leq \ln s$, we can find a trivial algorithm. This algorithm is also given in [FGG⁺23]: Add all nodes $v \in V$ to V_{out} such that $x_0(v) \geq 1/t$. Every $u \in U$ is covered by this; each u has at least one neighbor $v \in N(u)$ with $x_0(v) \geq 1/t$. Then, $|V_{out}|$ is at most t -times larger than $\sum_{v \in V} x_0(v)$, and $\sum_{v \in V} x_0(v) \leq 2 \cdot OPT$. Thus, this algorithm returns a $2 \ln s$ -approximate solution in $O(\log s \cdot \log t)$ rounds.

We can safely assume that $t > \ln s$. Thus, we can rewrite the round complexity of local rounding:

$$O(\log s) \cdot O(\log^2 t) \cdot \text{poly}(1/\epsilon)$$

Adding up all procedures the algorithm uses, Algorithm 3 has the round complexity of

$$O(\log s \cdot \log^2 t + \log^* n) \cdot \text{poly}(1/\epsilon)$$

4 Weighted Set Cover

In the last section, we demonstrated an improvement over the approximation ratio of an unweighted set cover algorithm given in [FGG⁺23]. However, adapting Algorithm 3 to support weights only yields an $O(\log(sW))$ -approximate algorithm, where W is an upper bound on the weights. In this section, our purpose is to create a weighted set cover algorithm with the same approximation ratio as the unweighted case, $(1 + (2 + \epsilon)\lceil \ln s \rceil)$. Theorem 1.2 is proved in this section.

Theorem 1.2. *There is a deterministic distributed algorithm in the LOCAL model that computes a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate weighted set cover in $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon) + O(\log t \log W)$ rounds, where the maximum set weight is at most W ($W \geq \max_{v \in V} w(v)$), each set has at most s elements ($s \geq \max_{v \in V} \deg(v)$), each element is contained in at most t sets ($t \geq \max_{u \in U} \deg(u)$), and $0 < \epsilon \leq 1$.*

A corollary can be drawn from this theorem for the unweighted case of the problem.

Corollary 1.3. *There is a deterministic distributed algorithm in the LOCAL model that computes a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate set cover in $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon)$ rounds, where each set has at most s elements ($s \geq \max_{v \in V} \deg(v)$) and $0 < \epsilon \leq 1$.*

This corollary provides an alternative algorithm for the unweighted set cover problem, alongside Algorithm 3 from the previous section. The two algorithms have different round complexities: Algorithm 3 has $O(\log s \cdot \log^2 t) \cdot \text{poly}(1/\epsilon)$, whereas the new algorithm has $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon)$. Consequently, the new algorithm is faster on instances where s and t are large. Bipartite graphs where $s = \Theta(n)$ and $t = \Theta(n)$ can be given as an example of such instances.

The weighted set cover algorithm is given in Algorithm 4. The approach is similar to that of the other deterministic algorithms given in Section 3. An outline is given here: We start by obtaining a 2-approximate weighted set cover solution for the input graph G , using the weighted version of the approximation algorithm from [KMW06]. Then, we cluster G such that the maximum diameter of a cluster is $O(\sqrt{\log n})$ and each node in $U \sqcup V$ has neighbors in at most $f(n) = 2^{\tilde{O}(\sqrt{\log n})}$ different clusters. We use Theorem 2.5, presented in Section 2.3, for this purpose. We utilize this clustering to compute an $O(\epsilon^5 / (f(n) \log n))$ -integral fractional weighted set cover solution, namely $\vec{x}$. Then, we run the local rounding procedure τ times; in each iteration, we round $\vec{x}$ to a binary solution. We collect the nodes $v \in V$ that are "selected" (i.e., their x_v value rounded to $x'_v = 1$) and add them to the covering set. After the local rounding procedure, there might be some uncovered elements $u \in U$ left. For each such u , we add a neighbor $v \in N(u)$ to the covering set such that v has the minimum weight among the nodes in $N(u)$. From this outline, the important difference between Algorithm 4 and Algorithm 3 can be seen: In Algorithm 4, nodes $v \in V$ calculate the $1/K$ -integral fractional solution, $\vec{x}$, using the information gathered from

their clusters. This gathering process takes $O(\sqrt{\log n})$ rounds, as clusters can have $O(\sqrt{\log n})$ diameter at most. Clustering of G itself has $\tilde{O}(\log^2 n)$ round complexity. In Algorithm 3, each $v \in V$ obtains its x_v value using $x_0(v)$, a value already stored locally in v , in $O(1)$ rounds. The following subsections provide the details of Algorithm 4.

Algorithm 4 Deterministic $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -Approximate Weighted Set Cover

Input: A bipartite graph $G = (U \sqcup V, E, w)$ where $w : V \rightarrow [1, \infty)$, parameter $\epsilon \in (0, 1]$
Output: A set $V_{out} \subseteq V$ with $N(V_{out}) = U$

- 1: $\vec{x}_0 : V \rightarrow [0, 1]$ is a 2-approx. fractional weighted set cover of [KMW06]
- 2: Cluster G with $\alpha = \sqrt{\log(N)}$, $f(n) = 2^{\tilde{O}(\sqrt{\log n})}$ using Theorem 2.5 (Theorem 2.1 of [GG23])
- 3: For every $v \in V$, compute x_v within the cluster
- 4: For every $u \in U$, define $N^*(u) \subseteq N(u)$ such that $\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \leq \sum_{v \in N^*(u)} x_v \leq \frac{2\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000}$
- 5: $d \leftarrow 1 + (1 - \frac{\epsilon}{100}) (1 - \frac{\epsilon}{10}) (\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000})$
- 6: $\tau \leftarrow \lceil 1/\ln d \rceil \lceil \ln s \rceil$
- 7: Compute 2-hop coloring of G with $\zeta = n^2$ colors using Linial's algorithm [Lin87]
- 8: $U_0 \leftarrow U$
- 9: Let $m_u = \min_{v \in N(u)} w(v)$, for all $u \in U$
- 10: **for** $i \leftarrow 0, \dots, \tau - 1$ **do**
- 11: $\mathbf{u}(\vec{x}) \leftarrow \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \sum_{v \in N^*(u)} \tilde{x}_v \right) + 2 \sum_{v \in V} w(v) x_v$
- 12: $\mathbf{c}(\vec{x}) \leftarrow \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} \tilde{x}_v \tilde{x}_{v'} \right) + \sum_{v \in V} w(v) \tilde{x}_v$
- 13: Plug in $\vec{x} = \vec{x}$, use Lemma 2.4 (Lemma 2.5 of [FGG⁺23]) with $\mathbf{u}, \mathbf{c}, \zeta, \delta = \frac{\epsilon}{100}, \mu = \frac{1}{2}, \lambda_{\min} = O\left(\frac{\epsilon^5}{f(n)\log n}\right)$, to round $\vec{x}$ to $\vec{x}'_i$
- 14: $V_{i+1} \leftarrow \{v \in V \mid x'_i(v) = 1\}$
- 15: $U_{i+1} \leftarrow U \setminus N(V_1 \cup V_2 \dots \cup V_{i+1})$
- 16: **end for**
- 17: Define V_{rem} by letting each element $u \in U_\tau$ choose its neighbor with the least weight
- 18: **return** $V_{out} = V_1 \cup \dots \cup V_\tau \cup V_{rem} \subseteq V$

4.1 Computing 1/K-Integral Solution with Clustering

In line 1, we compute a value for each $v \in V$, denoted by $x_0(v)$. These values, $\vec{x}_0$, form a 2-approximate fractional solution to the weighted variant of the set cover problem. We apply a similar step as in the previous deterministic algorithms presented, with the difference that we use a solution for the weighted version of the problem here. It should be noted that obtaining a 2-approximate fractional weighted solution has $O(\log(sW) \cdot \log t)$ round complexity ([KMW06]).

We create a partition of the graph $G = (U \sqcup V, E, w)$ using Theorem 2.5 in line 2. We select the parameter $\alpha = \sqrt{\log(N)}$ for the construction of clusters; the reason for this selection becomes clear in the discussion of the round complexity. We denote the partitioning with $P = \{P_1, \dots, P_k\}$. By the definition of clustering, the sets $P_1, \dots, P_k$ are pairwise disjoint and $\cup_{i=1}^k P_i = U \sqcup V$. Additionally, from the clustering conditions given in Theorem 2.5, $\max_{p \in P} \text{diam}(p) = O(\sqrt{\log n})$ and each node in $U \sqcup V$ has neighbors in at most $f(n) = 2^{\tilde{O}(\sqrt{\log n})}$ clusters. Using the partition P , we create an additional collection $V_P = \{V'_1, \dots, V'_k\}$ where $V'_i = V \cap P_i$ for all $1 \leq i \leq k$. V_P can be interpreted as the restriction of the partition P to the set V . Furthermore, we define a collection of subsets of U using V_P : $U' = \{U'_1, \dots, U'_k\}$

where $U'_i = N(V'_i)$. Clearly $\cup_{i=1}^k U'_i = U$. It should be noted that while $V'_i \cap V'_j = \emptyset$ for all $i \neq j$, U'_i and U'_j are not necessarily disjoint. Figure 4.1 gives a visualization for the collections introduced above.

In line 3, we establish a new fractional solution, $\vec{x}$, using the 2-approximate solution, $\vec{x}_0$. The motivation for creating a new solution is the same as the motivation for creating the $O(\epsilon^2/t)$ -integral solution in Algorithm 3, described in the last section: The local rounding procedure, when given fractional values $\vec{x}$ as input, has a worse round complexity if a fractional value in the vector $\vec{x}$ is arbitrarily close to 0. Using the clustering, our aim is to achieve a fractional solution which is $O(\epsilon^5/(f(n) \log n))$ -integral. Now, we describe how to achieve the fractional solution $\vec{x}$.

Nodes in $U \sqcup V$ only communicate within their clusters while establishing $\vec{x}$. In a cluster P_i , each $v \in V'_i$ sends its data (e.g., $x_0(v)$, $w(v)$, identifiers of its neighbors) to the other nodes in the cluster. After $O(\sqrt{\log n})$ rounds, each $v \in V'_i$ receives the information from all $v' \in V'_i$. We argue that all nodes $v \in V'_i$ can compute their new x_v value locally using only the information they gathered within the cluster P_i . We use the probabilistic method to prove that there exists a $O(\epsilon^5/f(n) \log n)$ -integral solution $\vec{x}$ along with the additional conditions.

We start by defining a subset for each $U'_i \in U'$. Let $U'_{i,b} \subseteq U'_i$ such that

$$U'_{i,b} = \left\{ u \in U'_i \mid \sum_{v \in N(u) \cap V'_i} x_0(v) \geq \epsilon/50f(n) \right\}$$

We can interpret $U'_{i,b}$ as follows: It contains only those $u \in U'_i$ that are partially covered at least a certain amount by the nodes in V'_i . Next, we create a random variable X_v for all $v \in V$. The rules for how we assign a value to X_v for each $v \in V$ are given below:

- If $x_0(v) \geq \frac{1}{kf(n) \log n}$, then $X_v = \frac{\epsilon}{10} \cdot x_0(v)$
- If $x_0(v) < \frac{1}{kf(n) \log n}$
 - If $v \in V'_i$ where $U'_{i,b} \neq \emptyset$,
 - * With probability $p = kf(n) \log n x_0(v)$, $X_v = \frac{\epsilon}{10kf(n) \log n}$
 - * With probability $1 - p$, $X_v = 0$
 - If $v \in V'_i$ where $U'_{i,b} = \emptyset$, then $X_v = 0$

It can be inferred from this selection process that all X_v are independent. The constant k depends on ϵ (specifically, $k = \Theta(1/\epsilon^4)$), and its precise value will be determined in the proof of Claim 4.2. We prove that the collection of random variables X_v in each cluster satisfies some conditions with non-zero probability.

Claim 4.1. *For all $V'_i \in V_P$,*

$$\Pr \left[\sum_{v \in V'_i} w(v) X_v \leq \left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v) x_0(v) \right] \geq \frac{\epsilon}{20 + \epsilon}$$

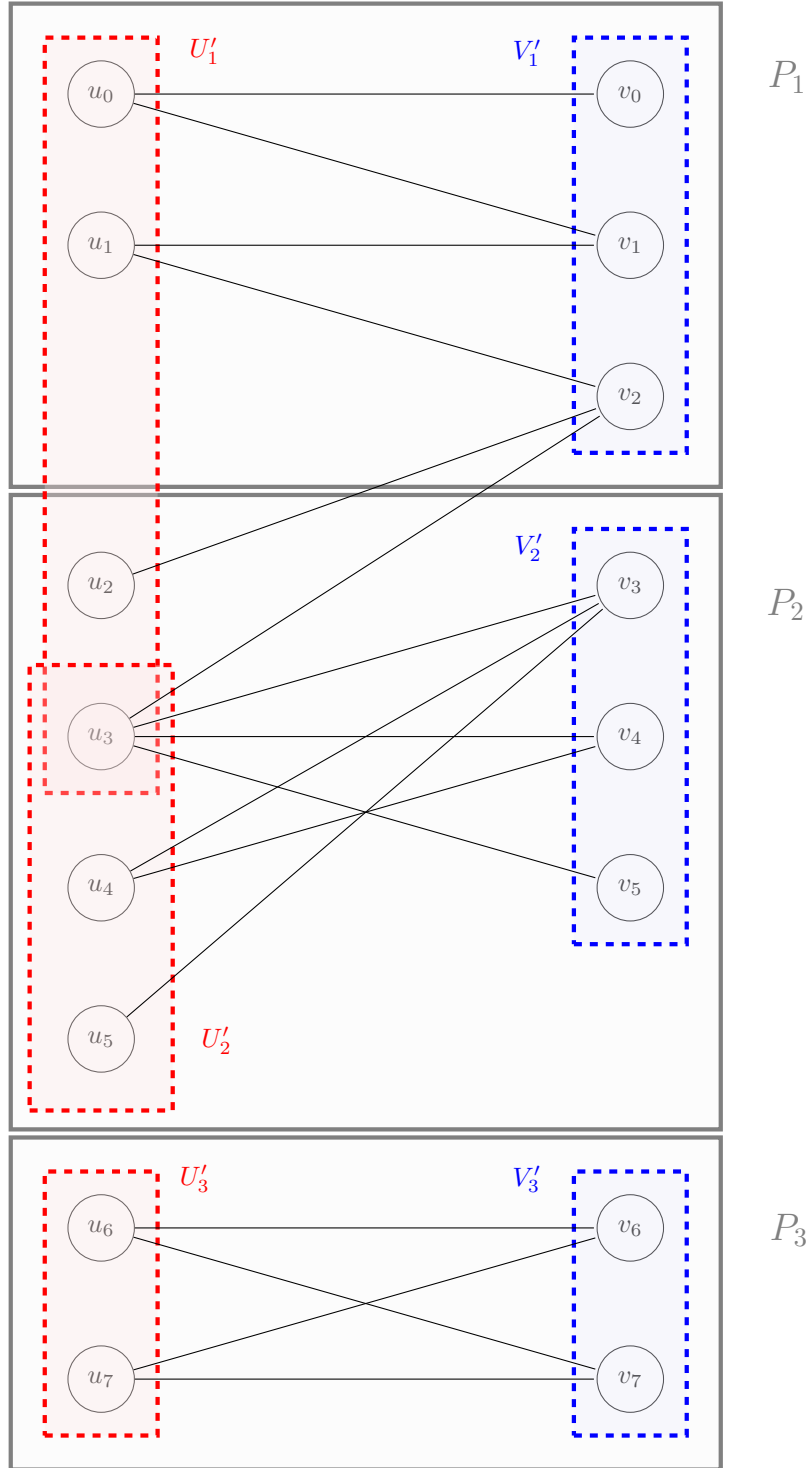


Figure 4.1: A partitioning $P = \{P_1, P_2, P_3\}$ is given for a bipartite graph G . $V_P = \{V'_1, V'_2, V'_3\}$ where $V'_1 = \{v_0, v_1, v_2\}$, $V'_2 = \{v_3, v_4, v_5\}$ and $V'_3 = \{v_6, v_7\}$. $U' = \{U'_1, U'_2, U'_3\}$ where $U'_1 = \{u_0, u_1, u_2, u_3\}$, $U'_2 = \{u_3, u_4, u_5\}$ and $U'_3 = \{u_6, u_7\}$.

Proof. For all V'_i with $U'_{i,b} = \emptyset$, it is trivial to obtain the bound. Let $V'_{i,s} \subseteq V'_i$ such that $v \in V'_{i,s}$ if $x_0(v) < 1/(kf(n) \log n)$. Then,

$$\sum_{v \in V'_i} w(v)X_v = \sum_{v \in V'_{i,s}} w(v)X_v + \sum_{v \in V'_i \setminus V'_{i,s}} w(v)X_v = \sum_{v \in V'_i \setminus V'_{i,s}} \frac{\epsilon}{10} \cdot w(v)x_0(v) \leq \sum_{v \in V'_i} \frac{\epsilon}{10} \cdot w(v)x_0(v)$$

For the sets V'_i with $U'_{i,b} \neq \emptyset$, we use Markov's inequality:

$$\begin{aligned} \Pr \left[\sum_{v \in V'_i} w(v)X_v \leq \left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v) \right] &\geq 1 - \Pr \left[\sum_{v \in V'_i} w(v)X_v \geq \left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v) \right] \\ &\geq 1 - \frac{\mathbb{E}[\sum_{v \in V'_i} w(v)X_v]}{\left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v)} \\ &= 1 - \frac{\left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v)}{\left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v)} \\ &= 1 - \frac{1}{1 + \frac{\epsilon}{20}} \\ &= \frac{\epsilon}{20 + \epsilon} \end{aligned}$$

In the third step, we use $\mathbb{E}[\sum_{v \in V'_i} w(v)X_v] = \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v)$. We finish the proof by showing this equality:

$$\begin{aligned} \mathbb{E} \left[\sum_{v \in V'_i} w(v)X_v \right] &= \sum_{v \in V'_i} \mathbb{E}[w(v)X_v] \\ &= \sum_{v \in V'_i} w(v)\mathbb{E}[X_v] \\ &= \sum_{v \in V'_{i,s}} w(v)\mathbb{E}[X_v] + \sum_{v \in V'_i \setminus V'_{i,s}} w(v)\mathbb{E}[X_v] \\ &= \sum_{v \in V'_{i,s}} w(v) \left(p \cdot \frac{\epsilon}{10kf(n) \log n} + (1-p) \cdot 0 \right) + \sum_{v \in V'_i \setminus V'_{i,s}} \frac{\epsilon}{10} \cdot w(v)x_0(v) \\ &= \sum_{v \in V'_{i,s}} w(v) \left(kf(n) \log nx_0(v) \cdot \frac{\epsilon}{10kf(n) \log n} \right) + \sum_{v \in V'_i \setminus V'_{i,s}} \frac{\epsilon}{10} \cdot w(v)x_0(v) \\ &= \sum_{v \in V'_{i,s}} \frac{\epsilon}{10} \cdot w(v)x_0(v) + \sum_{v \in V'_i \setminus V'_{i,s}} \frac{\epsilon}{10} \cdot w(v)x_0(v) \\ &= \sum_{v \in V'_i} \frac{\epsilon}{10} \cdot w(v)x_0(v) \end{aligned}$$

□

Claim 4.2. For all $V'_i \in V_P$ and for all $u \in U'_{i,b}$,

$$\Pr \left[\sum_{v \in V'_i \cap N(u)} X_v \geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i \cap N(u)} x_0(v) \right] \geq 1 - \frac{\epsilon}{8n^3}$$

Proof. Let $V_{i,u} = V'_i \cap N(u)$ and let $V_{i,u}^s \subseteq V_{i,u}$ be the set of vertices v such that $v \in V_{i,u}$ and $x_0(v) < 1/(kf(n) \log n)$. Let $S = \sum_{v \in V_{i,u}^s} X_v$. The expectation of S is given by

$$\mathbb{E}[S] = \sum_{v \in V_{i,u}^s} \left(kf(n) \log n x_0(v) \cdot \frac{\epsilon}{10kf(n) \log n} \right) = \sum_{v \in V_{i,u}^s} \frac{\epsilon}{10} \cdot x_0(v)$$

We choose $r = \frac{\epsilon}{20} \sum_{v \in V_{i,u}} \frac{\epsilon}{10} \cdot x_0(v)$. Since $r \geq \frac{\mathbb{E}[S]\epsilon}{20}$, we can invoke Corollary 2.2 with $b = \frac{\epsilon}{10kf(n) \log n}$ and $\omega = \frac{20}{\epsilon}$:

$$\begin{aligned} \Pr[|S - \mathbb{E}[S]| \geq r] &\leq 2 \exp \left(-\frac{r}{3\omega b} \right) \\ &= 2 \exp \left(-\frac{r}{\frac{3 \cdot 20 \cdot \epsilon}{\epsilon \cdot 10kf(n) \log n}} \right) \\ &= 2 \exp \left(-\frac{rkf(n) \log n}{6} \right) \\ &\leq 2 \exp \left(-\frac{kf(n) \log n \cdot \frac{\epsilon^2}{200} \sum_{v \in V_{i,u}} x_0(v)}{6} \right) \\ &\leq 2 \exp \left(-\frac{kf(n) \log n \cdot \frac{\epsilon^2}{200} \cdot \frac{\epsilon}{50f(n)}}{6} \right) \\ &\leq 2 \exp \left(-\frac{k \log n \cdot \epsilon^3}{6 \cdot 10^4} \right) \end{aligned}$$

Substituting $k = \frac{42 \cdot 10^4}{\epsilon^4}$:

$$\begin{aligned} \Pr[|S - \mathbb{E}[S]| \geq r] &\leq 2 \exp \left(-\frac{k \log n \cdot \epsilon^3}{6 \cdot 10^4} \right) \\ &= 2 \exp \left(-\frac{7}{\epsilon} \log n \right) \\ &= \frac{2}{n^{\frac{7}{\epsilon}}} \\ &\leq \frac{2\epsilon}{n^7} \\ &\leq \frac{\epsilon}{8n^3} \end{aligned}$$

Assuming $|S - \mathbb{E}[S]| \leq r$:

$$\begin{aligned}
\sum_{v \in V_{i,u}} X_v &= \sum_{v \in V_{i,u} \setminus V_{i,u}^s} X_v + S \geq \sum_{v \in V_{i,u} \setminus V_{i,u}^s} \frac{\epsilon}{10} \cdot x_0(v) + \mathbb{E}[S] - r \\
&= \sum_{v \in V_{i,u}} \frac{\epsilon}{10} \cdot x_0(v) - r \\
&= \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V_{i,u}} x_0(v)
\end{aligned}$$

□

According to the previous two claims, random variables X_v satisfy some properties for clusters with non-zero probability. Now, our aim is to prove that there exists an $O(\epsilon^5/f(n) \log n)$ -integral solution with the properties shown in Claim 4.1 and 4.2 for every cluster.

Claim 4.3. *There exists a $\vec{x}$ such that the following conditions hold:*

1. For all $v \in V$,

$$x_v \in \{0\} \cup \left[\left(\frac{\epsilon^5}{42 \cdot 10^5 \cdot f(n) \log n} \right), 1 \right]$$

2. For all $V'_i \in V_P$,

$$\sum_{v \in V'_i} w(v) x_v \leq \left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v) x_0(v)$$

3. For all $V'_i \in V_P$ such that $U'_{i,b} \neq \emptyset$, and for all $u \in U'_{i,b}$

$$\sum_{v \in V'_i \cap N(u)} x_v \geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i \cap N(u)} x_0(v)$$

Proof. We use the random variable X_v to argue that there is a non-zero probability of sampling X_v values such that all conditions above hold.

First, we show the third condition holds with high probability for the random variable X_v for an arbitrary cluster $V'_i \in V_P$ with $U'_{i,b} \neq \emptyset$. We fix a cluster $V'_i \in V_P$ with $|U'_{i,b}| = m$, where $m \in \mathbb{Z}^+$ and $m \leq n$. For each $u \in U'_{i,b}$, let $\mathcal{E}_{i,u}$ be the event that

$$\sum_{v \in V'_i \cap N(u)} X_v < \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i \cap N(u)} x_0(v)$$

From Claim 4.2, we know that $\Pr[\mathcal{E}_{i,u}] \leq \epsilon/8n^3$. Here, we are interested in the probability that none of

the bad events $\mathcal{E}_{i,u}$ occur for all $u \in U'_{i,b}$. This probability can be written as:

$$\begin{aligned} \Pr \left[\overline{\bigcup_{u \in U'_{i,b}} \mathcal{E}_{i,u}} \right] &= 1 - \Pr \left[\bigcup_{u \in U'_{i,b}} \mathcal{E}_{i,u} \right] \\ &\geq 1 - \sum_{u \in U'_{i,b}} \Pr[\mathcal{E}_{i,u}] \\ &\geq 1 - \frac{m\epsilon}{8n^3} \\ &\geq 1 - \frac{\epsilon}{8n^2} \end{aligned}$$

We showed the second condition holds for the random variable X_v for every $V'_i \in V_P$ with probability $\epsilon/(20 + \epsilon)$ in Claim 4.1. Let $\mathcal{E}_2$ denote the event that the second condition is violated, and $\mathcal{E}_3$ the event that the third condition is violated. For an arbitrary cluster $V'_i \in V_P$ with $U'_{i,b} \neq \emptyset$, under the assumption that $n \geq 2$, we have:

$$\begin{aligned} \Pr[\overline{\mathcal{E}_2 \cup \mathcal{E}_3}] &= 1 - \Pr[\mathcal{E}_2 \cup \mathcal{E}_3] \\ &\geq 1 - \Pr[\mathcal{E}_2] - \Pr[\mathcal{E}_3] \\ &\geq 1 - \left(1 - \frac{\epsilon}{20 + \epsilon}\right) - \frac{\epsilon}{8n^2} \\ &= \frac{\epsilon}{20 + \epsilon} - \frac{\epsilon}{8n^2} \\ &> 0 \end{aligned}$$

Since the probability is non-zero, there exists an assignment of values $\vec{x}$ within each such cluster that simultaneously satisfies the second and third conditions. Moreover, since every sample of X_v is $O(\epsilon^5/(f(n) \log n))$ -integral, the first condition is automatically satisfied. Finally, because the clusters V'_i are pairwise disjoint, the random choices made within different clusters are independent. We can therefore apply the above argument to each cluster independently, obtaining a global assignment $\vec{x}$ that satisfies all three conditions simultaneously. $\square$

We proved that there exists a fractional solution $\vec{x}$ with the given conditions in Claim 4.3, and the value x_v can be obtained by every $v \in V$ in $O(\sqrt{\log n})$ rounds. Every $v \in V$ computes x_v locally after receiving all information from the nodes $v' \in V$ which share a cluster with v .

In line 4, we create a subset $N^*(u) \subseteq N(u)$ for all $u \in U$. Similarly to Algorithm 3, these subsets $N^*(u)$ are used to select nodes $v \in N^*(u)$ to cover u in local rounding. To prove that there exists an $N^*(u)$ for every $u \in U$ satisfying the conditions specified in line 4, we prove a preliminary claim first.

Claim 4.4. *For every $u \in U$,*

$$\sum_{v \in N(u)} x_v \geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(1 - \frac{\epsilon}{50}\right)$$

Proof. We start by summing x_v of the nodes $v \in N(u)$ which reside only in those clusters V'_i such that

$u \in U'_{i,b}$.

$$\begin{aligned}
\sum_{v \in N(u)} x_v &\geq \sum_{i|u \in U'_{i,b}} \sum_{v \in V'_i \cap N(u)} x_v \\
&\geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{i|u \in U'_{i,b}} \sum_{v \in V'_i \cap N(u)} x_0(v) \\
&= \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(\sum_{V'_i \in V_P} \sum_{v \in V'_i \cap N(u)} x_0(v) - \sum_{i|u \notin U'_{i,b}} \sum_{v \in V'_i \cap N(u)} x_0(v) \right) \\
&= \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(\sum_{v \in N(u)} x_0(v) - \sum_{i|u \notin U'_{i,b}} \sum_{v \in V'_i \cap N(u)} x_0(v) \right)
\end{aligned}$$

In the second inequality, the third condition of Claim 4.3 is used. We then rewrite the sum by taking the sum over all clusters $V'_i \in V_P$ and subtracting the contributions from clusters where $u \notin U'_{i,b}$. This is equivalent to the summation of $x_0(v)$ values from all $V'_i \cap N(u)$ such that $u \in U'_{i,b}$.

$$\begin{aligned}
\sum_{v \in N(u)} x_v &\geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(\sum_{v \in N(u)} x_0(v) - \sum_{i|u \notin U'_{i,b}} \sum_{v \in V'_i \cap N(u)} x_0(v) \right) \\
&\geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(1 - f(n) \cdot \frac{\epsilon}{50f(n)} \right) \\
&= \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(1 - \frac{\epsilon}{50} \right)
\end{aligned}$$

Here, we substitute $\sum_{v \in N(u)} x_0(v)$ with 1 since $\vec{x}_0$ is a fractional solution and every node $u \in U$ is covered (i.e., $\sum_{v \in N(u)} x_0(v) \geq 1$). Moreover, according to the definition of $U'_{i,b}$, $\sum_{v \in V'_i \cap N(u)} x_0(v) < \epsilon/50f(n)$ if $u \notin U'_{i,b}$. The number of clusters $V'_i \in V_P$ with $u \in N(V'_i)$ cannot exceed $f(n)$ for every $u \in U$; the second condition of the clustering theorem, Theorem 2.5, implies that every $u \in U$ can have neighbors in at most $f(n)$ clusters. $\square$

Now, we establish the bounds for the subsets $N^*(u)$.

Claim 4.5. *There exists a subset $N^*(u) \subseteq N(u)$ for all $u \in U$, such that*

$$\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \leq \sum_{v \in N^*(u)} x_v \leq \frac{2\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000}$$

Proof. The set $N^*(u)$ can be created by adding nodes from $N(u)$. From Claim 4.4, we can add enough nodes to $N^*(u)$ such that

$$\sum_{v \in N^*(u)} x_v \geq \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \left(1 - \frac{\epsilon}{50}\right) = \frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000}$$

Then, since $x_v \leq \epsilon/10$ for all $v \in V$,

$$\begin{aligned} \frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} &\leq \sum_{v \in N^*(u)} x_v \leq \frac{\epsilon}{10} + \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \right) \\ &= \frac{2\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \end{aligned}$$

□

Lastly, we establish an upper bound on the total weight of the fractional solution $\vec{x}$.

Claim 4.6.

$$\sum_{v \in V} w(v)x_v \leq \left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \cdot 2 \cdot OPT$$

Proof. According to the second condition of Claim 4.3, for all $V'_i \in V_P$

$$\sum_{v \in V'_i} w(v)x_v \leq \left(1 + \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10}\right) \sum_{v \in V'_i} w(v)x_0(v)$$

We can conclude the proof by noting $\sum_{v \in V} w(v)x_0(v) \leq 2 \cdot OPT$ as $\vec{x}_0$ is a 2-approximate fractional solution. □

4.2 Rounding $1/K$ -Integral Solution

After line 4, our aim in the rest of the algorithm is to obtain a binary solution from the $O(\epsilon^5/(f(n) \log n))$ -integral solution $\vec{x}$. We use the local rounding procedure for this purpose, the same approach we take in Algorithm 3.

In lines 5 and 6, we assign values to d and τ , the adjustment constant and the number of times the local rounding routine runs, respectively. The reasoning behind the selected values becomes clear when we analyze the algorithm in the next subsections.

We need to select appropriate utility $\mathbf{u}$ and cost $\mathbf{c}$ functions for the local rounding procedure. We use the altered versions of the optimizations we used in Algorithm 3. Since we study the weighted version of the set cover problem in this section, the alterations to the optimizations reflect this difference. Previously, the second optimization goal is to minimize the number of selected nodes $v \in V$. For the weighted version, our corresponding aim is to minimize the total weight of the selected nodes in $v \in V$. This optimization can be represented by $\sum_{v \in V} w(v)\tilde{x}_v$ ($\sum_{v \in V} \tilde{x}_v$ is used for the unweighted version). The first optimization goal is to maximize $|U_i \setminus U_{i+1}|$ for each round i . We represented this optimization by the terms

$$\sum_{u \in U_i} \left(\sum_{v \in N(u)} \tilde{x}_v - \frac{1}{2} \sum_{v \in N(u)} \sum_{v' \neq v \in N(u)} \tilde{x}_v \tilde{x}_{v'} \right)$$

for the unweighted algorithm. We can reuse this optimization without change; after all, we still want to maximize the number of covered elements $u \in U$ every round. Adding weights to $v \in V$ should not

change this optimization, since the values $\tilde{x}_v$ of $v \in N(u)$ determine whether $u \in U$ is covered or not. However, we cannot use these two expressions together. The first optimization gives a result based on the summation of $\tilde{x}_v$ values; the second optimization, on the other hand, gives a result based on pairwise sums of $w(v)\tilde{x}_v$. Depending on the weights $\vec{w}$, the result we get can be heavily skewed towards the first or second optimization. We propose a different expression for the maximization of $|U_i \setminus U_{i+1}|$, incorporating the weights of the nodes:

$$\sum_{u \in U_i} \min_{v'' \in N(u)} w(v'') \left(\sum_{v \in N(u)} \tilde{x}_v - \frac{1}{2} \sum_{v \in N(u)} \sum_{v' \neq v \in N(u)} \tilde{x}_v \tilde{x}_{v'} \right)$$

For each $u \in U$, let $m_u = \min_{v \in N(u)} w(v)$. We use this shorthand to keep the expressions compact. The expressions for the two optimizations are now determined. We can distribute these expressions to the utility and cost functions:

$$\begin{aligned} \mathbf{u}(\vec{x}) &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \sum_{v \in N^*(u)} \tilde{x}_v \right) + 2 \sum_{v \in V} w(v) x_v \\ \mathbf{c}(\vec{x}) &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} \tilde{x}_v \tilde{x}_{v'} \right) + \sum_{v \in V} w(v) \tilde{x}_v \end{aligned}$$

We split the inner expression of the first optimization into two parts, $m_u \cdot \sum_{v \in N^*(u)} \tilde{x}_v$ and $m_u \cdot \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} \tilde{x}_v \tilde{x}_{v'}$, and distribute to the utility and cost functions, respectively. The minimization of $\sum_{v \in V} w(v) \tilde{x}_v$ is represented by the second term of the cost function. Further, we added the term $2 \sum_{v \in V} w(v) x_v$ to the utility function to satisfy the requirement for $\mathbf{u}$ and $\mathbf{c}$ according to Lemma 2.4. Next, we show that this requirement is satisfied by the selected $\mathbf{u}(\vec{x})$ and $\mathbf{c}(\vec{x})$:

Claim 4.7. $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) \geq \mathbf{u}(\vec{x})/2$

Proof. We prove the equivalent statement $\mathbf{c}(\vec{x}) \leq \mathbf{u}(\vec{x})/2$ instead. Both functions $\mathbf{u}$ and $\mathbf{c}$ consist of two terms each; we compare the corresponding terms individually. Comparing the first terms gives us

$$\begin{aligned} \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} &\leq \frac{1}{2} \sum_{v \in N^*(u)} x_v \left(\sum_{v' \in N^*(u)} x_{v'} \right) \\ &\leq \frac{1}{2} \sum_{v \in N^*(u)} x_v \cdot \left(\frac{2\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \right) \\ &\leq \frac{1}{2} \sum_{v \in N^*(u)} x_v \end{aligned}$$

The upper bound on $\sum_{v' \in N^*(u)} x_{v'}$ is used in the second inequality, given in Claim 4.5. This result implies

that

$$\begin{aligned} \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} \right) &\leq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \frac{1}{2} \sum_{v \in N^*(u)} x_v \right) \\ &= \frac{1}{2} \left(\frac{1}{d^{\tau-i}} \sum_{u \in U_i} \left(m_u \cdot \sum_{v \in N^*(u)} x_v \right) \right) \end{aligned}$$

This establishes the inequality for the first terms of the functions. Comparing the second terms is trivial:

$$\sum_{v \in V} w(v) x_v \leq \frac{1}{2} \left(2 \sum_{v \in V} w(v) x_v \right)$$

□

It is interesting to note that this requirement might not be satisfied by every valid fractional solution $\vec{x}$. However, we do not need a generalized result. We apply local rounding τ times, and each time we start rounding from the same values $\vec{x}$. Therefore, proving the requirement for $\mathbf{u}(\vec{x})$ and $\mathbf{c}(\vec{x})$ is sufficient. An analogous claim and its proof for the unweighted case can be found in Section 3.2, under Claim 3.4. Lastly, we emphasize an inequality derived in the previous proof. This is used in the analysis of the algorithm in the next subsection:

$$\begin{aligned} \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} &\leq \frac{1}{2} \sum_{v \in N^*(u)} x_v \cdot \left(\frac{2\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \right) \\ &= \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{2000} + \frac{\epsilon^3}{20000} \right) \sum_{v \in N^*(u)} x_v \end{aligned} \tag{9}$$

4.2.1 Analyzing One Round of Local Rounding

We have previously introduced $R_u(\vec{x}')$ in Section 3.2: It indicates whether $u \in U$ is covered by any $v \in N(u)$ given a binary solution $\vec{x}'$. We then proved in Section 3.2 that for any binary solution $\vec{x}'$,

$$R_u(\vec{x}') \geq \sum_{v \in N^*(u)} \tilde{x}'_v - \frac{1}{2} \sum_{v \in N^*(u)} \sum_{v' \neq v \in N^*(u)} \tilde{x}'_v \tilde{x}'_{v'}$$

For the rounded values $\vec{x}'$, which are obtained by rounding the solution $\vec{x}$ from line 3, we can derive the following using $\mathbf{u}$ and $\mathbf{c}$:

$$\begin{aligned} \mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}') - 2 \sum_{v \in V} w(v) x_v &\leq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u R_u(\vec{x}') - \sum_{v \in V} w(v) x'_v \\ &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i \setminus U_{i+1}} m_u - \sum_{v \in V_{i+1}} w(v) \\ &= \frac{1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - \sum_{u \in U_{i+1}} m_u \right) - S_{i+1} \end{aligned} \tag{10}$$

Here, we denote the sum of the weights of nodes $v \in V$ that are selected at the end of round i , $\sum_{v \in V_{i+1}} w(v)$, as S_{i+1} . Our aim is to lower-bound $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}')$ and show the progress of every round, following the same approach we used in the analysis of Algorithm 3. We start by finding a lower bound for $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})$:

$$\begin{aligned}
\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} w(v)x_v &= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \sum_{v \in N^*(u)} \left(x_v - \frac{1}{2} \sum_{v' \neq v \in N^*(u)} x_v x_{v'} \right) - \sum_{v \in V} w(v)x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \sum_{v \in N^*(u)} \left(x_v - \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{2000} + \frac{\epsilon^3}{20000} \right) \cdot x_v \right) - \sum_{v \in V} w(v)x_v \\
&= \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{10} + \frac{7\epsilon^2}{2000} - \frac{\epsilon^3}{20000} \right) \sum_{v \in N^*(u)} x_v - \sum_{v \in V} w(v)x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{10} + \frac{7\epsilon^2}{2000} - \frac{\epsilon^3}{20000} \right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \right) - \sum_{v \in V} w(v)x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{10} \right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} \right) - \sum_{v \in V} w(v)x_v \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{10} \right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} \right) - \left(1 + \frac{\epsilon}{20} \right) \left(\frac{2\epsilon}{10} \right) \cdot OPT \tag{11}
\end{aligned}$$

We use the definitions of $\mathbf{u}$ and $\mathbf{c}$ in the first equality. The first inequality is obtained by substituting the quadratic term with its upper bound using Eq. (9). In the second inequality, we replace $\sum_{v \in N^*(u)} x_v$ with its lower bound using Claim 4.5. In the third inequality, we substitute both cubic polynomials of ϵ with simpler polynomials. Specifically, the inequalities

$$\left(1 - \frac{\epsilon}{10} + \frac{7\epsilon^2}{2000} - \frac{\epsilon^3}{20000} \right) \geq \left(1 - \frac{\epsilon}{10} \right)$$

and

$$\left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} + \frac{\epsilon^3}{10000} \right) \geq \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000} \right)$$

can be easily verified for $0 < \epsilon \leq 1$. In the last inequality, we use Claim 4.6 to replace $\sum_{v \in V} w(v)x_v$.

In Lemma 2.4, it is stated that $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}')$ is lower bounded by $(1 - \delta)(\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}))$ with error parameter

$0 < \delta \leq 1$. Selecting $\delta = \epsilon/100$ gives us

$$\begin{aligned}
\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}') - 2 \sum_{v \in V} w(v)x_v &\geq \left(1 - \frac{\epsilon}{100}\right) \cdot (\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x})) - 2 \sum_{v \in V} w(v)x_v \\
&= \left(1 - \frac{\epsilon}{100}\right) \cdot \left(\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} w(v)x_v\right) - \left(\frac{2\epsilon}{100}\right) \sum_{v \in V} w(v)x_v \\
&\geq \left(1 - \frac{\epsilon}{100}\right) \cdot \left(\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} w(v)x_v\right) - \left(\frac{2\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \\
&\geq \left(1 - \frac{\epsilon}{100}\right) \cdot \left(\frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right) - \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT\right) \\
&\quad - \left(\frac{2\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \\
&\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right) \\
&\quad - \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \tag{12}
\end{aligned}$$

We use Claim 4.6 similarly as previously, in the second inequality. Then, $\mathbf{u}(\vec{x}) - \mathbf{c}(\vec{x}) - 2 \sum_{v \in V} w(v)x_v$ is replaced by its lower bound given in Eq. (11). Upper and lower bounds for the expression $\mathbf{u}(\vec{x}') - \mathbf{c}(\vec{x}') - 2 \sum_{v \in V} w(v)x_v$ are given in Eq. (10) and Eq. (12), respectively. Putting these bounds together:

$$\begin{aligned}
\frac{1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - \sum_{u \in U_{i+1}} m_u \right) - S_{i+1} &\geq \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right) \\
&\quad - \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT
\end{aligned}$$

Here, we select $d = 1 + \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)$. Rewriting the inequality above yields

$$\frac{1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - \sum_{u \in U_{i+1}} m_u \right) - S_{i+1} \geq \frac{(d-1)}{d^{\tau-i}} \sum_{u \in U_i} m_u - \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \tag{13}$$

4.3 Approximation Ratio and Round Complexity

We finish proving Theorem 1.2 in this section, by showing Algorithm 4 returns a $(1 + (2 + \epsilon)\lceil \ln s \rceil)$ -approximate solution to the weighted set cover problem in $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon) + O(\log t \cdot \log W)$ rounds. We start with the approximation ratio analysis. Then, we discuss the round complexity.

We define a potential function for every step of the rounding:

Definition 4.1. For all $0 \leq i \leq \tau$,

$$\Phi_i = \frac{1}{d^{\tau-i}} \sum_{u \in U_i} m_u + S_1 + \dots + S_i + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) (\tau - i) \cdot OPT$$

This potential function is an analog of the potential function introduced in Section 3.3 for the unweighted case. Φ_0 and Φ_τ represent the states before and after the local rounding subroutine, respectively. Other states Φ_i describe the states after the i th round of local rounding.

Our proof structure is the same as that used for Algorithm 3 in Section 3.3: We prove $\Phi_0 \leq (1 + (2 + \epsilon)\lceil \ln s \rceil) \cdot OPT$ first. Then, we show that the weight of the solution Algorithm 4 outputs is less than or equal to Φ_τ . Finally, the non-increasing monotonicity of Φ_i is proven; this implies $\Phi_\tau \leq \Phi_0$. These steps entail that

$$\sum_{v \in V_{out}} w(v) \leq (1 + (2 + \epsilon)\lceil \ln s \rceil) \cdot OPT$$

First, we examine Φ_0 :

$$\Phi_0 = \frac{1}{d^\tau} \sum_{u \in U} m_u + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \tau \cdot OPT$$

We need to select an appropriate τ . For $\tau = \lceil 1/\ln d \rceil \lceil \ln s \rceil$,

$$\begin{aligned} \Phi_0 &= \frac{1}{d^{\lceil 1/\ln d \rceil \lceil \ln s \rceil}} \sum_{u \in U} m_u + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\ &\leq \frac{1}{d^{\lceil \log_d s \rceil}} \sum_{u \in U} m_u + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\ &= \frac{1}{s} \sum_{u \in U} m_u + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \end{aligned}$$

We observe that

$$OPT \geq \frac{1}{s} \sum_{u \in U} m_u$$

A small proof is provided here: Suppose we have an optimal solution $S = \{v_1, \dots, v_k\}$. The nodes $v_i \in V$ cover all $u \in U$, i.e., $\bigcup_{i=1}^k N(v_i) = U$. We examine an arbitrary $v_i \in S$: Suppose v_i covers elements $u_1, \dots, u_{s'}$ where $s' \leq s$. Then,

$$w(v_i) \geq \frac{1}{s'} \sum_{u \in N(v_i)} \min_{v \in N(u)} w(v) \geq \frac{1}{s} \sum_{u \in N(v_i)} \min_{v \in N(u)} w(v)$$

Since S covers all $u \in U$:

$$\sum_{i=1}^k w(v_i) \geq \frac{1}{s} \sum_{i=1}^k \sum_{u \in N(v_i)} \min_{v \in N(u)} w(v) \geq \frac{1}{s} \sum_{u \in U} \min_{v \in N(u)} w(v) = \frac{1}{s} \sum_{u \in U} m_u$$

Thus, we can further simplify:

$$\begin{aligned}
\Phi_0 &\leq \frac{1}{s} \sum_{u \in U} m_u + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\
&\leq OPT + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil \cdot OPT \\
&= (1 + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil) \cdot OPT
\end{aligned}$$

Using the value of $d = 1 + \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)$,

$$\begin{aligned}
\Phi_0 &\leq (1 + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil) \cdot OPT \\
&\leq (1 + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left(1 + \frac{1}{\ln d}\right) \lceil \ln s \rceil) \cdot OPT \\
&= (1 + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left(1 + \frac{1}{\ln \left(1 + \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)\right)}\right) \lceil \ln s \rceil) \cdot OPT \\
&\leq (1 + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \left(1 + \frac{\left(1 + \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)\right)}{\left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)}\right) \lceil \ln s \rceil) \cdot OPT \\
&\stackrel{**}{\leq} (1 + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot \frac{20000 + 10000\epsilon}{2000\epsilon + 120\epsilon^2 + \epsilon^3} \lceil \ln s \rceil) \cdot OPT \\
&= (1 + (2 + \epsilon) \lceil \ln s \rceil) \cdot OPT
\end{aligned}$$

The second inequality follows from the logarithmic inequality:

$$\forall \alpha > 0. \quad \ln(\alpha) \geq 1 - \frac{1}{\alpha} = \frac{\alpha - 1}{\alpha}$$

given in [Top07]. The inequality denoted with (**) is proven in the appendix.

Next, our aim is to show that the total weight of the solution, $\sum_{v \in V_{out}} w(v)$, is at most Φ_τ :

$$\begin{aligned}
\sum_{v \in V_{out}} w(v) &= \sum_{v \in V_1 \cup \dots \cup V_\tau \cup V_{rem}} w(v) \\
&\leq \sum_{v \in V_1} w(v) + \dots + \sum_{v \in V_\tau} w(v) + \sum_{v \in V_{rem}} w(v) \\
&\leq \sum_{v \in V_1} w(v) + \dots + \sum_{v \in V_\tau} w(v) + \sum_{u \in U_\tau} m_u \\
&= S_1 + \dots + S_\tau + \sum_{u \in U_\tau} m_u \\
&= \Phi_\tau
\end{aligned}$$

Finally, non-increasing monotonicity of Φ_i is proven:

$$\begin{aligned}
\Phi_i - \Phi_{i+1} &= \frac{1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - d \cdot \sum_{u \in U_{i+1}} m_u \right) - S_{i+1} + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \\
&= \frac{1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - \sum_{u \in U_{i+1}} m_u \right) - S_{i+1} + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \\
&\quad - \frac{d-1}{d^{\tau-i}} \cdot \sum_{u \in U_{i+1}} m_u
\end{aligned}$$

We use Eq. (13) to finish the proof.

$$\begin{aligned}
\Phi_i - \Phi_{i+1} &= \frac{1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - \sum_{u \in U_{i+1}} m_u \right) - S_{i+1} + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT - \frac{d-1}{d^{\tau-i}} \cdot \sum_{u \in U_{i+1}} m_u \\
&\geq \frac{d-1}{d^{\tau-i}} \sum_{u \in U_i} m_u - \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT + \left(1 + \frac{\epsilon}{100}\right) \left(1 + \frac{\epsilon}{20}\right) \left(\frac{2\epsilon}{10}\right) \cdot OPT \\
&\quad - \frac{d-1}{d^{\tau-i}} \sum_{u \in U_{i+1}} m_u \\
&= \frac{d-1}{d^{\tau-i}} \sum_{u \in U_i} m_u - \frac{d-1}{d^{\tau-i}} \cdot \sum_{u \in U_{i+1}} m_u \\
&= \frac{d-1}{d^{\tau-i}} \left(\sum_{u \in U_i} m_u - \sum_{u \in U_{i+1}} m_u \right) \\
&\geq 0
\end{aligned}$$

We finish the proof of Theorem 1.2 with the round complexity analysis. Algorithm 4 starts with obtaining a 2-approximate fractional solution (line 1), which takes $O(\log(sW) \cdot \log t)$ rounds [KMW06]. Then, in lines 2-4, we obtain an $O(\epsilon^5/(f(n) \log n))$ -integral solution. This solution is calculated for every node $v \in V$ locally, after receiving information from nodes in their cluster. The maximum diameter of the clustering is $O(\sqrt{\log n})$, thus the clustering algorithm runs in $\tilde{O}(\log^2 n)$ rounds according to Theorem 2.5. Before applying local rounding, we compute a 2-hop coloring of G with $\zeta = n^2$ colors (line 7). The coloring runs in $O(\log^* n)$ rounds [Lin87]. After coloring, we run the local rounding procedure $\tau = \lceil 1/\ln d \rceil \lceil \ln s \rceil$ times in lines 10-16. Each round of local rounding takes

$$O\left(\frac{1}{\epsilon} \log^2 \left(\frac{f(n) \log n}{\epsilon^6}\right) + \log \left(\frac{f(n) \log n}{\epsilon^6}\right) \cdot \log^*(n^2)\right)$$

Using $f(n) = 2^{\tilde{O}(\sqrt{\log n})}$, we can rewrite the round complexity as

$$O\left(\frac{1}{\epsilon} \left(\tilde{O}(\sqrt{\log n}) + \log \left(\frac{1}{\epsilon^6}\right)\right)^2 + \left(\tilde{O}(\sqrt{\log n}) + \log \left(\frac{1}{\epsilon^6}\right)\right) \cdot \log^*(n^2)\right)$$

The factor $1/\epsilon$ dominates the polylogarithmic terms $\log^2(1/\epsilon^6)$ and $\log(1/\epsilon^6)$. Further, $\log^*(n^2)$ is domi-

nated by $\sqrt{\log n}$. Thus, we are left with

$$O\left(\frac{1}{\epsilon} \left(\tilde{O}(\sqrt{\log n})\right)^2\right) = O\left(\frac{1}{\epsilon} \left(\tilde{O}(\log n)\right)\right)$$

Since we have τ rounds of local rounding, the round complexity of the whole local rounding procedure is

$$\left\lceil \frac{1}{\ln d} \right\rceil \lceil \ln s \rceil O\left(\frac{1}{\epsilon} \left(\tilde{O}(\log n)\right)\right) = \frac{\lceil \ln s \rceil}{O(\ln(1+\epsilon))} O\left(\frac{1}{\epsilon} \left(\tilde{O}(\log n)\right)\right)$$

We can replace $O(\ln(1+\epsilon))$ with $O(\epsilon)$. At the end, we have the expression

$$\lceil \ln s \rceil \cdot \tilde{O}(\log n) \cdot \text{poly}\left(\frac{1}{\epsilon}\right)$$

The round complexity of Algorithm 4 can be represented by:

$$\tilde{O}(\log s \cdot \log n) \cdot \text{poly}\left(\frac{1}{\epsilon}\right) + \tilde{O}(\log^2 n) + O(\log(sW) \cdot \log t)$$

From this expression, the reasoning for the selection of $\alpha = \sqrt{\log N}$ can be seen. Any other selection can either increase the $\tilde{O}(\log^2 n)$ term or the $\tilde{O}(\log s \cdot \log n)$ term, which would lead to a worse complexity. Since $s, t \leq N$, we can simplify the round complexity using $\log s = O(\log n)$ and $\log t = O(\log n)$:

$$\tilde{O}(\log^2 n) \cdot \text{poly}\left(\frac{1}{\epsilon}\right) + O(\log t \cdot \log W)$$

5 Discussion

In this section, we review our results and discuss potential improvements for future work.

We designed an unweighted set cover algorithm which has an approximation ratio of $1 + (2 + \epsilon)\lceil \ln s \rceil$ and round complexity of $O(\log s \cdot \log^2 t + \log^* n) \cdot \text{poly}(1/\epsilon)$. This is an improvement, in terms of the approximation ratio, over the algorithm given by Faour et al. [FGG⁺23] where the approximation ratio is $O(\log s)$. Our algorithm is slower by only a factor of $\text{poly}(1/\epsilon)$. Further, we constructed a weighted set cover algorithm, which combines the ideas of low-diameter clustering and local rounding. These two ideas have been used previously for problems such as $O(1)$ -approximate maximum matching and MIS [GG23]. The weighted set cover algorithm runs in $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon) + O(\log t \cdot \log W)$ rounds, with an approximation ratio of $1 + (2 + \epsilon)\lceil \ln s \rceil$. From the weighted set cover algorithm, we can obtain an unweighted set cover algorithm by setting all weights to $w(v) = 1$. This unweighted set cover algorithm has $\tilde{O}(\log^2 n) \cdot \text{poly}(1/\epsilon)$ round complexity.

Recall that the randomized Algorithm 1 achieves an expected approximation ratio of $(1 + 2\lceil \ln s \rceil)$. Our deterministic algorithms derandomize this process via local rounding and achieve a ratio of $(1 + (2 + \epsilon)\lceil \ln s \rceil)$. By adjusting the ϵ parameter, we can have an approximation ratio arbitrarily close to $(1 + 2\lceil \ln s \rceil)$. This shows that our deterministic algorithms preserve the solution quality of the randomized algorithm.

5.1 Future Work

This subsection presents ideas that could potentially improve our algorithms but have not yet been fully explored.

For Algorithm 4, one of the bottlenecks appears to be the clustering subroutine. To obtain a faster algorithm, we need to reduce the factor $\tilde{O}(\log^2 n)$ in the round complexity. One potential approach that can reduce the round complexity is to use a faster clustering algorithm. In [GG23], an alternative clustering algorithm is given:

Theorem 5.1 (Theorem 2.2 of [GG23]). *Let $\alpha \leq \log n$ be an arbitrary value and assume that each node $v \in V$ is equipped with a value $x_v \in [\frac{1}{n}, 1]$. There exists a deterministic distributed algorithm that in $\tilde{O}(\alpha \log n)$ rounds of the LOCAL model computes a partition C with*

1. *diameter $O(\alpha)$ and*
2. *let $V^{\text{good}} = \{v \in V \mid \deg_C(v) = 2^{\tilde{O}(\log n)/\alpha}\}$, then we have $\sum_{v \in V^{\text{good}}} x_v \geq 0.9 \sum_{v \in V} x_v$*

This clustering algorithm achieves a better round complexity at the cost of allowing a small fraction of nodes to have neighbors in more than $2^{\tilde{O}(\log n)/\alpha}$ clusters. If we can compensate for these nodes, we could achieve a weighted set cover algorithm with a round complexity of $\tilde{O}(\log^{5/3} n) \cdot \text{poly}(1/\epsilon) + O(\log(sW) \cdot \log t)$ by selecting $\alpha = \log^{2/3}(N)$. From this algorithm, we can obtain an unweighted set cover algorithm that runs in $\tilde{O}(\log^{5/3} n) \cdot \text{poly}(1/\epsilon) + O(\log s \cdot \log t)$ rounds.

The unweighted set cover algorithm obtained from Algorithm 4 has a better worst-case round complexity than Algorithm 3, as it involves fewer logarithmic factors. However, the logarithmic factors in Algorithm 3 are based on s and t (except for an additive $O(\log^* n)$), rather than n . Thus, Algorithm 3 is expected to run faster on bipartite graphs where $s \ll n$ and $t \ll n$. Replacing some of the $O(\log n)$ factors in Algorithm 4 with $O(\log s)$ or $O(\log t)$ could make Algorithm 4 faster on such bipartite graphs. To achieve this, we could potentially tailor the clustering algorithm given in Section 2.3 to bipartite graphs, substituting $\log n$ factors with $\log s$ or $\log t$.

Finally, eliminating the $\log W$ factor from the round complexity of Algorithm 4 would make the complexity independent of weights. This factor is introduced solely during the computation of the initial 2-approximate fractional weighted set cover solution. It is important to note that the round complexity of the rest of Algorithm 4 (lines 2-18) is already independent of W . Therefore, designing a 2-approximate fractional weighted set cover algorithm whose round complexity is independent of W would eliminate the $\log W$ factor entirely.

Bibliography

- [ALGP89] B. Awerbuch, M. Luby, A.V. Goldberg, and S.A. Plotkin. Network decomposition and locality in distributed computation. In *30th Annual Symposium on Foundations of Computer Science*, pages 364–369, 1989.
- [CKW10] Graham Cormode, Howard Karloff, and Anthony Wirth. Set cover algorithms for very large datasets. In *Proceedings of the 19th ACM International Conference on Information and Knowledge Management, CIKM '10*, page 479–488, New York, NY, USA, 2010. Association for Computing Machinery.
- [DKM19] Janosch Deurer, Fabian Kuhn, and Yannic Maus. Deterministic distributed dominating set approximation in the congest model. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC '19*, page 94–103, New York, NY, USA, 2019. Association for Computing Machinery.
- [DS14] Irit Dinur and David Steurer. Analytical approach to parallel repetition. STOC '14, New York, NY, USA, 2014. Association for Computing Machinery.
- [EGM18] Guy Even, Mohsen Ghaffari, and Moti Medina. Distributed set cover approximation: primal-dual with optimal locality. *Leibniz International Proceedings in Informatics (LIPIcs)*, 121:22–1, 2018.
- [FGG⁺23] Salwa Faour, Mohsen Ghaffari, Christoph Grunau, Fabian Kuhn, and Václav Rozhoň. *Local Distributed Rounding: Generalized to MIS, Matching, Set Cover, and Beyond*, pages 4409–4447. 2023.
- [FGK17] Manuela Fischer, Mohsen Ghaffari, and Fabian Kuhn. Deterministic distributed edge-coloring via hypergraph maximal matching. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 180–191. IEEE, 2017.
- [Fis20] Manuela Fischer. Improved deterministic distributed matching via rounding. 33(3–4):279–291, June 2020.
- [GG23] Mohsen Ghaffari and Christoph Grunau. Faster deterministic distributed mis and approximate matching. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023*, page 1777–1790, New York, NY, USA, 2023. Association for Computing Machinery.
- [GGH⁺23] Mohsen Ghaffari, Christoph Grunau, Bernhard Haeupler, Saeed Ilchi, and Václav Rozhoň. Improved distributed network decomposition, hitting sets, and spanners, via derandomization.

- In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2532–2566. SIAM, 2023.
- [GGR21] Mohsen Ghaffari, Christoph Grunau, and Václav Rozhoň. Improved deterministic network decomposition. In *Proceedings of the Thirty-Second Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '21, page 2904–2923, USA, 2021. Society for Industrial and Applied Mathematics.
 - [GJ79] M.R. Garey and D.S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-completeness*. Mathematical Sciences Series. W. H. Freeman, 1979.
 - [GK22] Mohsen Ghaffari and Fabian Kuhn. Deterministic distributed vertex coloring: Simpler, faster, and without network decomposition. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1009–1020. IEEE, 2022.
 - [Har19] David G. Harris. Distributed local approximation algorithms for maximum matching in graphs and hypergraphs. In *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 700–724, 2019.
 - [HKP01] Michal Hanckowiak, Michal Karonski, and Alessandro Panconesi. On the distributed complexity of computing maximal matchings. *SIAM Journal on Discrete Mathematics*, 15(1):41–57, 2001.
 - [HMN22] Magnús M. Halldórsson, Yannic Maus, and Alexandre Nolin. Fast Distributed Vertex Splitting with Applications. In Christian Scheideler, editor, *36th International Symposium on Distributed Computing (DISC 2022)*, volume 246 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1–26:24. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022.
 - [KMW06] Fabian Kuhn, Thomas Moscibroda, and Roger Wattenhofer. The price of being near-sighted. In *SODA 2006: 17th ACM-SIAM Symposium on Discrete Algorithms, Miami, Florida, USA*, 2006.
 - [KMW16] Fabian Kuhn, Thomas Moscibroda, and Roger Wattenhofer. Local computation: Lower and upper bounds. *Journal of the ACM (JACM)*, 63(2):1–44, 2016.
 - [KV12] B. Korte and J. Vygen. *Combinatorial Optimization: Theory and Algorithms*. Algorithms and Combinatorics. Springer Berlin Heidelberg, 2012.
 - [Lin87] Nathan Linial. Distributive graph algorithms global solutions from local data. In *Proceedings of the 28th Annual Symposium on Foundations of Computer Science*, SFCS '87, page 331–335, 1987.
 - [MU17] Michael Mitzenmacher and Eli Upfal. *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press, 2017.
 - [Pel00] David Peleg. *Distributed computing: a locality-sensitive approach*. Society for Industrial and Applied Mathematics, 2000.

- [RG20] Václav Rozhoň and Mohsen Ghaffari. Polylogarithmic-time deterministic network decomposition and distributed derandomization. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2020, page 350–363, New York, NY, USA, 2020. Association for Computing Machinery.
- [Top07] Flemming Topsøe. Some bounds for the logarithmic function. *Inequality theory and applications*, 4(01), 2007.

A Appendix

We prove the inequality (*), given in Section 3.3, first:

Claim A.1 (*). For $0 < \epsilon \leq 1$,

$$\frac{200 + 100\epsilon}{20\epsilon + \epsilon^2} \geq 1 + \frac{\left(1 + \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)\right)}{\left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)}$$

Proof. Since $(20\epsilon + \epsilon^2)$ and $\left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)$ are both positive:

$$\begin{aligned} (200 + 100\epsilon) \cdot \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right) &\geq (20\epsilon + \epsilon^2) \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right) \\ &\quad + (20\epsilon + \epsilon^2) \cdot \left(1 + \left(1 - \frac{\epsilon}{20}\right) \left(\frac{\epsilon}{10} - \frac{3\epsilon^2}{250}\right)\right) \end{aligned}$$

By representing the polynomials on both sides in ascending powers of ϵ , we have

$$20\epsilon + \frac{33\epsilon^2}{5} - \frac{79\epsilon^3}{50} + \frac{3\epsilon^4}{50} \geq 20\epsilon + 5\epsilon^2 - \frac{12\epsilon^3}{25} - \frac{\epsilon^4}{100} + \frac{3\epsilon^5}{2500}$$

Collecting all terms to one side, and factoring out ϵ^2 gives us:

$$\frac{8}{5} - \frac{11\epsilon}{10} + \frac{7\epsilon^2}{100} - \frac{3\epsilon^3}{2500} \geq 0$$

We finish the proof by observing that the positive terms dominate the negative terms:

$$\frac{8}{5} + \frac{7\epsilon^2}{100} \geq \frac{11}{10} + \frac{3\epsilon^2}{2500} \geq \frac{11\epsilon}{10} + \frac{3\epsilon^3}{2500}$$

□

Using similar proof steps, we prove (**) given in Section 4.3:

Claim A.2 (**). For $0 < \epsilon \leq 1$,

$$\frac{20000 + 10000\epsilon}{2000\epsilon + 120\epsilon^2 + \epsilon^3} \geq 1 + \frac{1 + \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)}{\left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)}$$

Proof. We clear the denominators by multiplying each term by a suitable positive term. Then, we can

work with the numerators:

$$(20000 + 10000\epsilon) \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right) \geq (2000\epsilon + 120\epsilon^2 + \epsilon^3) \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right) \\ + (2000\epsilon + 120\epsilon^2 + \epsilon^3) \left(1 + \left(1 - \frac{\epsilon}{100}\right) \left(1 - \frac{\epsilon}{10}\right) \left(\frac{\epsilon}{10} - \frac{7\epsilon^2}{1000}\right)\right)$$

We expand both sides of the inequality in ascending powers of ϵ :

$$2000\epsilon + 640\epsilon^2 - \frac{813\epsilon^3}{5} + \frac{214\epsilon^4}{25} - \frac{7\epsilon^5}{100} \geq 2000\epsilon + 520\epsilon^2 - 47\epsilon^3 - \frac{16\epsilon^4}{25} + \frac{181\epsilon^5}{1250} + \frac{3\epsilon^6}{50000} - \frac{7\epsilon^7}{500000}$$

Collecting all terms to one side and factoring ϵ^2 from the resulting polynomial gives us:

$$120 - \frac{578\epsilon}{5} + \frac{46\epsilon^2}{5} - \frac{537\epsilon^3}{2500} - \frac{3\epsilon^4}{50000} + \frac{7\epsilon^5}{500000} \geq 0$$

120 is clearly greater than $578\epsilon/5$ for all $0 < \epsilon \leq 1$. Comparing $46\epsilon^2/5$ with the sum of absolute values of the remaining negative terms finishes the proof:

$$\frac{46\epsilon^2}{5} \geq 9\epsilon^2 \geq \epsilon^3 + \epsilon^4 \geq \frac{537\epsilon^3}{2500} + \frac{3\epsilon^4}{50000}$$

□